Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Medieninformatik

Abschlussarbeit

zur Erlangung des akademischen Grades

Master of Science

Calibrating LLM based Code Generation

Vorgelegt von	Robin Kuschnereit
am	21. Juli 2025
Referent	Prof. Dr. Adrian Ulges
Korreferentin	Viola Campos

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich versichere, dass ich KI-Tools lediglich als Hilfsmittel verwendet habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich verantworte die Übernahme jeglicher von mir verwendeter KI-generierter Inhalte vollumfänglich selbst. Die jeweiligen Benutzungseingaben (z.B. Prompts) sind der Arbeit beigefügt.

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Erklärung zur Verwendung der Masterthesis

Hiermit erkläre ich mein Einverständnis mit den im folgenden aufgeführten Verbreitungsformen dieser Abschlussarbeit:

Verbreitungsform	**Ja**	**Nein**
Veröffentlichung des Titels der Arbeit im Internet	×	
Veröffentlichung der Arbeit im Internet		×

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Abstract

This work addresses the calibration of code generated by large language models (LLM). Calibration is the process of adjusting the predicted confidence values to more accurately reflect the true likelihood of correctness. A well-calibrated model has a strong relationship between the likelihood of correctness and the model's confidence. This work investigates different approaches to estimate the confidence of generated code samples. One category uses the average token probability. Another category uses an evaluator LLM to assess the code correctness. This is done quantitatively, where the model is prompted for a numerical confidence value, and qualitatively, where the model is prompted for a confidence value from a textual scale. Each value on this scale corresponds to a predefined level of confidence. Experiments are conducted using the humaneval dataset with the MultiPL-E benchmark. This allows the the original python tasks to be translated into 22 different programming languages. The results show that the code samples with an average token probability are not well calibrated out of the box. The evaluator LLM achieves better calibration than the average token probabilities. Furthermore multiple approaches were evaluated to improve calibration. The Methods used are histogram binning, linear regression, iterative group histogram binning, and iterative group linear binning. The simpler methods, histogram binning and linear regression produced the best results. Histogram binning achieved an ECE of 0.036 and an ASCE of 0.003. Linear regression achieved a MSE value of 0.207. The more sophisticated methods, iterative group histogram binning and iterative group linear binning, achieved better calibration. However they are not as effective as the simpler methods. This may be because they are overfitting. Calibration in specific groups was also explored. The **simple** and **combined** grouping style delivered the best results here. The **simple** grouping style achieved an ASCE of 0.004. The **combined** grouping style achieved an ECE of 0.053, a MSE of 0.205 and therefore a skill score of 0.077. Furthermore, histogram binning and linear regression achieved the best group calibration. With some exceptions iterative group linear binning was on par with the simpler methods. In summary, the calibration improved with all approaches used. It is unexpected that the more complex methods delivered worse results than the simpler ones. Next steps could include exploring different baselines than the average token probability, using more data, and exploring new groups.

Contents

Chapter 1

Introduction

In recent years large language models or LLM for short, have shown notable abilities in a wide range of tasks. One of these tasks include automated code generation. Automated code generation is the process of leveraging an LLM or other frameworks to generate code with high-level input. For LLM, this is a written explanation of the code's functions or the code itself. Models like Chat-GPT, Qwen-Coder or Gemini can produce code snippets, finish functions and solve complex programming problems. Studies show that these models are already used by software developers [18] and another study finds that approximately 31% of postdocs use AI-assistants like Chat-GPT [9]. For instance the developer can utilize the model to generate or refine code through chat interactions. Furthermore the developer can try to fix faulty code by prompting the model. Vaithilingam et al. [18] show that generated code might not be perfect, but it can be used as a good starting point for further development steps. The AI-assistant builds the rough framework and the developer refines the generated code. The term sample is used in this work to describe the generated code. Showing the existing interest in using LLMs and particular in using them to generate code.

These models are inherently unreliable and often hallucinate. Therefore a human needs to check the results. A reliable confidence metric would help a human decide if the code is functioning and useful. This metric could then be used to identify possible faulty code. One way to create such a metric is the usage of the individual token confidence values. They can be returned by the model besides the generated textual outputs. This confidence value displays the models probability that a specific token appears at its place in the generated token sequence. An average value of this token probabilities is used in this work as an indicator for the correctness of the generated code. Another possible way to get a confidence value for the generated code is to use an evaluator LLM. This work evaluates two approaches that involve an evaluator LLM: quantitative and qualitative. The quantitative approach asks the evaluator LLM to return a numerical confidence value and the qualitative approach asks the evaluator LLM for a textual value from a predefined scale. In the postprocessing this scale assigned a numerical confidence value to the sample. The confidence value however, that is created by one of these approaches, might not be a good indicator to find out if the generate code is functioning or if the model might have made some mistakes [11]. Such mistakes could be unreachable code, logical errors or security issues. Therefore calibration could be a solution to make the confidence values more reliable.

Although the quality of the generated code has steadily improved, calibration is a critical yet often overlooked aspect. Calibration is the task of adjusting the model's confidence in its generated output and the true likelihood of the generated code being correct. Meaning that if a sample has a confidence of

50% it is estimated to have a probability of 50% to be correct. Furthermore the topic of multicalibration is even less researched. Multicalibration is the process of calibrating multiple (intersecting) subsets. These subsets are referred to as groups in this work. Possible groups include prompts that are longer than a given length or contain examples. Since LLMs have a broad range of applications, it is expected that the calibration will vary across different areas.

Calibration helps to build more trustworthy and transparent systems which allow the user to identify the usefulness of the generated code. Furthermore a reliable uncertainty measure allows the developer to take countermeasures and verify the generated code. The focus of this work lies on the subject of Code Generation.

Figure 1.1 displays the process of generating code via an LLM. It can be seen that LLM has a confidence of 90% that the generated code is correct. However, if calibration is used the probability of correctness drops to 30%. Without calibration, the programmer would assume that the generated code is correct and can be used without caution. With calibration, however, the programmer would be more suspicious of the generated piece of code and would examine it more closely to determine its reliability.

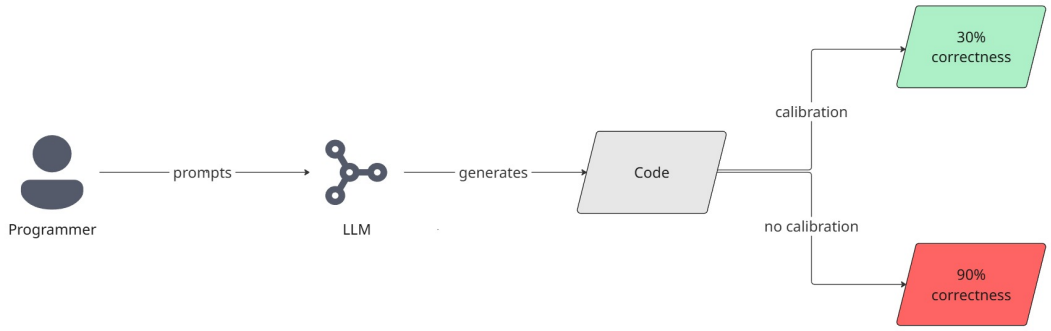


Figure 1.1: Illustration of the code generation process. A programmer prompts and LLM with a request to generate code. The LLM processes the request and output the code with the probability that the code is 90% correct. This percentage is misleading, because with calibration the code is only 30% correct.

Calibration plays a crucial role in code generation. It helps the developer to better assess the generated code pieces. Faulty code may lead to system failures, security vulnerabilities or maintenance burdens. A well calibrated model does not only provide helpful code generations but also a reliable metric which determines the uncertainty of the model. This metric should represent how confident the model is in the correctness of its generated code. It is assumed that the confidence score or the token likelihoods are not sufficient, because the models tend to overestimate or underestimate their results. This assumption is confirmed in this thesis and should be improved by using calibration. To achieve this different calibration methods are used. These methods were already used by Detommaso et al. [6]. In their case they used these methods for hallucination detection on question-answer datasets.

This thesis aims to explore the calibration properties of LLM on code generation. The goal of this work is to investigate methods which assess and improve the calibration of confidence values from an LLM. Furthermore different approaches to retrieve an initial guess of the models confidence for code generations are shown. These approaches include: the average token probability and an evaluator LLM which asses the correctness of generated code pieces. This evaluator LLM uses two approaches: qualitative and quantitative. This confidence should represent the expected probability correctness

after the calibration approaches were applied.

This thesis' contributions are:

- The realization of the calibration and multicalibration approaches onto the subject area code generation. As to the date of this work no other work evaluated multicalibration on LLM based code generations.
- Furthermore, the subject of groups for generated code is explored. These groups are used in the multicalibration approaches. Here is displayed which groups are useful to better the calibration on code generations. To create these groups the metadata of the generated code is used.
- Another difference is the fact that the topic of correctness prediction is evaluated, instead of hallucination what Detommaso et al. [6] did in their work.

This work is structured as follows. In the first chapter 2 the foundations for our work are explained. This includes the basics of LLM, calibration, important metrics and the calibration methods that are used. Furthermore works that are related to our topic are displayed. The next chapter 3 covers an overview over our used components. These components include the data generation, evaluation of code with a LLM and the calibration approaches. Then the evaluation chapter 4 displays the results of our work. Therefore the used datasets, benchmarks and models are shown. Furthermore experiments with the different approaches are conducted. This experiments will all answer a question. In the discussion chapter 5 sets the results into context and will display potential weaknesses. Besides that some options for further research are given.

Chapter 2

Foundations

In the following chapter the foundations that are needed for this work are displayed. Beginning with the explanation of LLMs. The training and then the generation process are briefly showcased. Furthermore the metrics that are used to evaluate the calibration on our data are formally explained. Therefore an example of bad and well calibrated data are shown. After this each scores for the example are calculated. The next part looks briefly into the used methods to improve calibration on the data. In the end related work on calibration is shown.

2.1 Large Language Models

The concept of a large language model is a fundamental part of this work. Therefore the origin of these models and how their generations process works are presented. Before the creation of the large language models, language models, were used [24]. These language models relied heavily on statistics. Their capabilities were enhanced through the research of transformers [19], especially autoregressive transformers. These transformers are based on multi-head attention. Multi-head attention enables the model to focus on different parts of the input simultaneously and learn contextual relationships. It helps the model understand complex patterns and relationships within the data. Autoregressive transformers use the before generated sequence to predict the next most likely token. Text is converted into numerical tokens and these tokens are then converted into vectors with the help of word embedding. This allows the transformer to learn the context of tokens. Todays LLMs are pretrained LLMs and have hundreds of billions of parameters that are learned of massive amounts of text data. For example the DeepSeek R1 model has 671B parameters [27].

Figure 2.1 displays where the LLM, that are used in this work, are located in the field of artificial intelligence (AI) [18].

- First the Figure 2.1 shows a big sphere of AI. This sphere represents the discipline of imitating human behavior with machines to solve tasks without explicit instructions.
- Machine learning is a subset of AI. Machine learning is used to create a model for given input data. This model can then be used to make predictions on new data. Typical algorithms for machine learning are linear regression or support vector machines.
- Deeper into machine learning, deep learning can be found. Deep learning models are made of simulated neurons which can be arranged in multiple layers. Each of these models have an

input layer and an output layer. There may be several hidden layers between the input and output layer. The layers transport information form the input to the output layer. These deep learning models are trained on large datasets. The training works with the backpropagation algorithm. The main advantage of deep learning against machine learning is that deep learning can use unstructured data.

- The last subset in Figure 2.1 shows generative DL. In this case that consists of LLMs, which are used for code generation. The generative DL takes an input and can create new content.

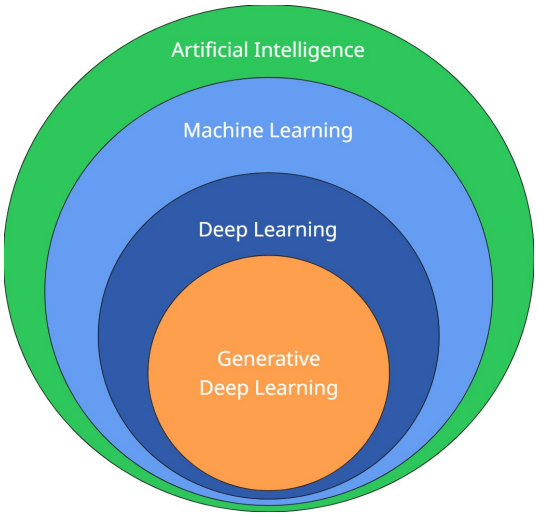


Figure 2.1: The image shows in which subject area generative deep learning is placed. LLMs belong to generative DL. The image is inspired by Amol Wagh [31]

The goal of an LLM is to predict the probability of a sequence of tokens $y_1, ..., y_m$ given an input sequence of tokens $x_1, ..., x_n$[22]. x represents the input prompt and y represents the generated token sequence. These tokens can be words, sub words or characters. Each has a set of these tokens that are used to represent text. This set is called vocabulary. After the tokenization each token is mapped to an integer ID. That number can be used to look up the corresponding embedding vector. The probability of such a token sequence is calculated with the Equation 2.1. For a sequence of tokens this results in a chain of conditional probabilities.

$$P(y_{1:m}|x_{1:n}) = P(y_1|x_{1:m}) \times P(y_2|y_1, x_{1:m}) \times ... \times P(y_m|y_{1:m}, x_{1:m}) \tag{2.1}$$

To generate new tokens the user inputs text into the LLM. This text is then converted into a sequence of input tokens $x_1, ..., x_n$. The generation process is also called decoding, because it uses the decoding part of a transformer. This decoding process can be adjusted with multiple parameters. The context tokens can also be called „prompt“. When the LLM gets the prompt it starts to predict the next token. The model generates new tokens until the maximal token count or an end token is reached. The maximal token count can be defined in the models configuration. For the next token the model can either choose the most probable token or consider multiple candidates. Without any randomization the output of such a model would always be the same, given the same input [22]. This is also known as greedy search. Greedy search selects the token with the highest conditional probability of the vocabulary. Therefore it would be deterministic. This is good for tasks where the outcome is critical, but a disadvantage for tasks where creativity and flexibility are preferred.

Tong Xiao and Jingbo Zhu [22] show two sampling methods in their work. These sampling methods help getting variation into the generated outputs of LLMs. The first sampling method is the Top-k sampling. This method chooses the next k most probable tokens during each step of the generation process. After the k tokens are selected the model regenerates the probabilities for these tokens and selects the token with the highest probability. The other sampling method is the Top-p sampling. In contrast to the Top-k sampling, Top-p sampling does not choose from a fixed size if tokens. Instead it searches for the smallest subset of tokens that have a cumulative probability greater then the value of p. These methods create a balance between the randomness of the output and the ability to output reasonable sentences.

Another parameter that controls the randomness of the generated output is called temperature [22]. Equation 2.2 shows the softmax function to convert the output logits into probabilities. $P(t_n)$ is the probability of the n-th token in the vocabulary and l_n is the logit of this token. The sum $\sum_i e^{l_i/t}$ displays all input values. The logit l_n and l_i are divided by the temperature value t. If a higher value is chosen for this parameter then the difference between the output logits will decrease. That has the consequence that the tokens have a more equal opportunity to be selected. Therefore the output has more variation. A lower temperature value has the effect of making the outputs more reliable and deterministic. Making it more likely that highly probable tokens will be chosen.

$$P(t_n) = \frac{e^{l_n/t}}{\sum_i e^{l_i/t}} \tag{2.2}$$

as by Matthew Renze and Erhan Guven [14].

This thesis works with the probabilities that come out of Equation 2.2. These probabilities lay the baseline for our experiments.

2.2 Calibration

A model is considered well calibrated when the accuracy is equal to the confidence of the model [8]. Accuracy is the percentage of correct classifications. In this work, accuracy refers to the probability that the generated code samples are correct. For example calibration plays a role in weather forecast [30]. The forecasted weather probabilities are calibrated with past observations. Therefore they can reliably adjust the forecast probabilities that the model makes.

Calibration is important because it helps the end user to rely on the output probabilities. It is not practical to measure calibration over a continuous numerical value. The problem is that probabilities are a continuous numerical value. To counter this problem a uniform grid is used to which the continuous numerical values are assigned.

Figure 2.2 shows the reliability diagram for 4 samples. The model that outputs the confidence for these samples is well calibrated. The x-axis shows the confidence value that was predicted by the model for each bin. In total the diagram shows 11 bins. The y-axis shows the correctness of the samples in each bin. The bin with the confidence 50% contains two samples. One sample is correct and the other one is incorrect. Resulting in a 50% correctness of said bin and is therefore considered calibrated. Of course, it would be better if the correct sample had a confidence of 100% and the incorrect sample had a confidence of 0%. Given the core idea that the samples are grouped by confidence and mapped to new confidences. The perfect calibration for every bin is symbolized through the blue dotted line. The other two samples are in the bin with 100% and 0%. The one with 100% is correct and the sample in bin 0% is incorrect. This means every bin is perfectly calibrated and therefore the model is well calibrated.

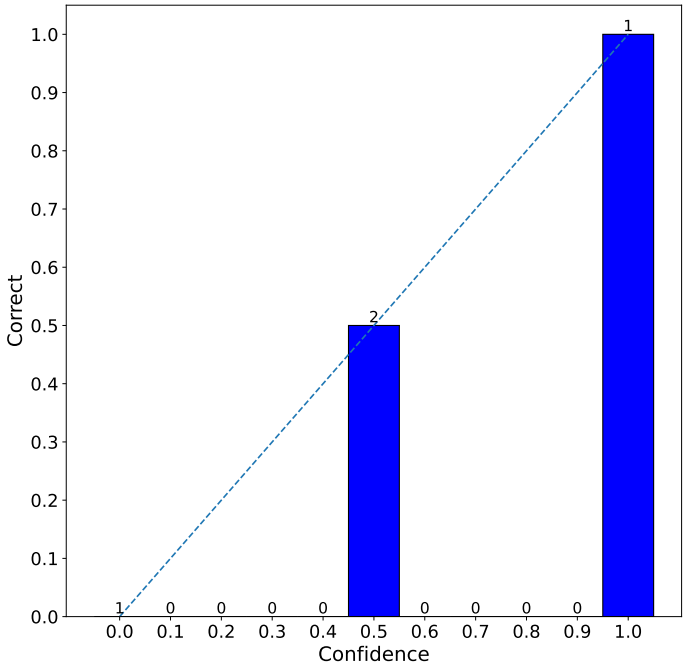


Figure 2.2: A reliability chart that displays the model confidence on the x-axis and the average correctness per bin on the y-axis. This is an example of good calibrated data. A total of four samples can be seen. Bin with a 50% confidence contains two samples. One correct and one incorrect. The dotted line indicates perfect calibration and every bin with samples meets the line.

On the other side the Figure 2.3 shows the output of a hypothetical model which outputs poorly calibrated confidence values. The bin with a 50% confidence is still well calibrated, but the bin 0% and 100% are badly calibrated. That because the sample in bin 0% is correct and the whole bin has now a correctness of 100%. An the opposite is the case for the bin 100%. The sample is incorrect and that is why the correctness in the bin is 0%.

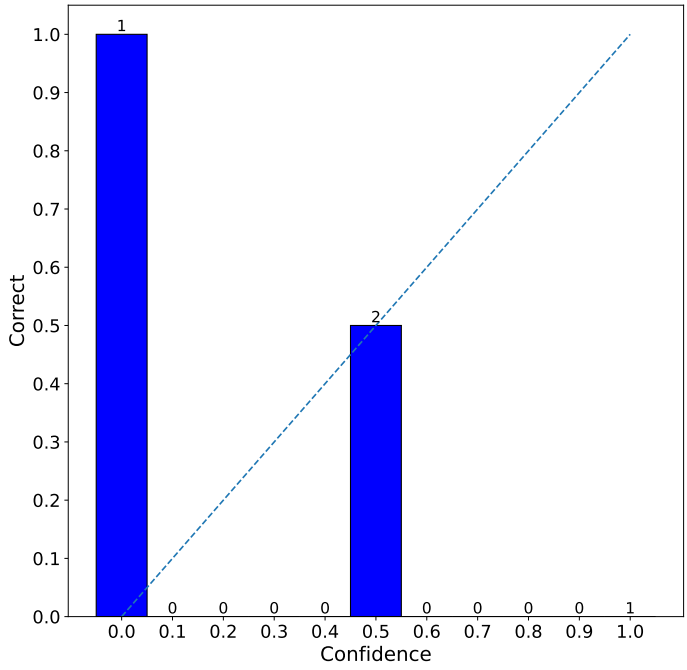


Figure 2.3: A reliability chart that displays the model confidence on the x-axis and the average correctness per bin on the y-axis. This is an example of bad calibrated data. A total of four samples can be seen. Bin with a 50% confidence contains two samples. One correct and one incorrect. This bin is calibrated. The dotted line indicates perfect calibration. Nevertheless the bin 0% has a correct sample and is therefore poorly calibrated.

Multicalibration

Furthermore this work will look into the topic of multicalibration with the presented approaches. Multicalibration was introduced by the work of Hébert-Johnson et al. [26]. Multicalibration tries to calibrate multiple subsets of data at the same time. This enables adjusting the calibration in each subset, which could have a different calibration out of the box. The subsets are called groups later on and can also be intersecting.

Figure 2.4 shows the same sample as Figure 2.2. There are still only four samples, but this time the samples are assigned to groups. Two samples are in the red group and two samples are in the blue group. One blue sample is in the bin 50% and is incorrect. That is why no bar is shown. The other one is in the 100% and is correct. Therefore the sample is in the right bin. In the red group a correct sample is located in the bin 50% and an incorrect sample is located in the 0% bin. Overall splitting the samples into groups gives another perspective on calibration. The well calibrated example from Figure 2.2 is poorly calibrated pertaining to groups. Multicalibration is the approach of calibrating these subsets (groups) of data at the same time.

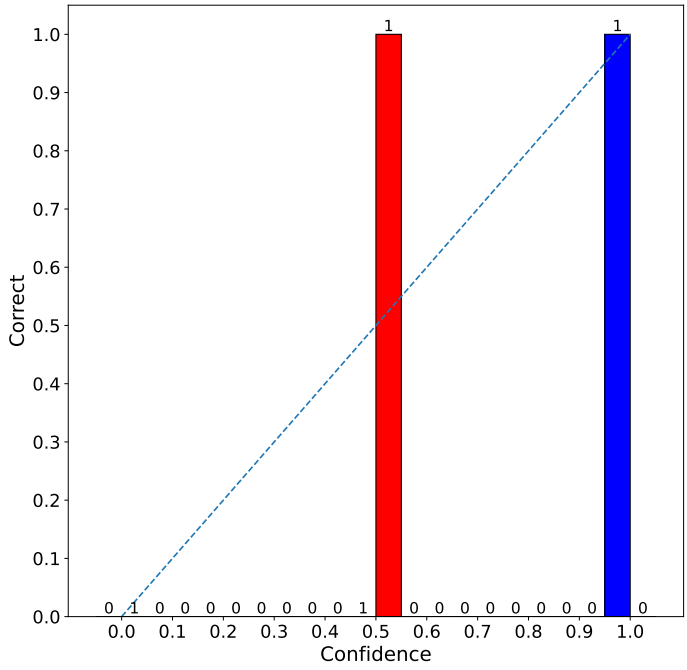


Figure 2.4: A reliability chart that displays the model confidence on the x-axis and the average correctness per bin on the y-axis. This is an example of different calibrated groups and is the example from Figure 2.2 with the extension of groups. A total of four samples can be seen. Two samples are in the red group and two samples are in the blue group. Both groups are not well calibrated. Because one blue sample is in the 50% bin, but is incorrect and should therefore be in the 0% bin. The correct red sample in bin 50% should be in the 100% bin. The dotted line indicates perfect calibration.

2.3 Metrics

In the previous section visualizations of calibration were shown. Additionally to quantitatively measure the degree of calibration of a model the following metrics are used.

2.3.1 ECE

The first score is the expected calibration error (ECE) [12]. This score measures the difference between confidence and correctness per bin S_b. The bins are defined by $[\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, ..., \frac{m-1}{m}, 1\}$. The count of bins is defined by the number m. S_b contains all samples that are assigned to the bin b. All samples in a bin have the same confidence value. How this bin assignment works will be explained in Section 3.3.1. Each bin is weighted by the share of samples P_b in this bin. The pair (x, y) describes the sample x and the label y. The label y indicates if the sample is correct. $f(x)$ is the confidence that the model f has in the correctness of the sample x. The lower the value of the ECE is the better is the calibration of the model. It should be noted that the ECE value is influenced by the size of the bins.

$$ECE = \sum_{b=1}^{n} P_b \times |E[y - f(x)|(x,y) \in S_b]|, \text{ where } b \in [\frac{1}{m}] \tag{2.3}$$

For the example of good calibration in the Section 2.2 above the ECE value would be 0, because each bin has no difference between confidence value and correctness. The bad calibrated example on the other hand has a ECE value of 0.5, because $\frac{1}{4} \times 1 + \frac{2}{4} \times 0.5 + \frac{1}{4} \times 0$. It is still not that high because the two samples in bin 50% are still perfectly calibrated.

2.3.2 ASCE

Another metric that is used in this work is the average squared calibration error (ASCE) [6]. It is similar to the previous ECE score. The difference here is that the difference between confidence and correctness per bin S_b is squared and does not use the absolute value. An ASCE of 0 means that the model f is calibrated. The higher this score gets the worse is the calibration of the model.

$$ASCE = \sum_{b=1}^{n} P_b \times (E[y - f(x)|(x,y) \in S_b])^2, \text{ where } b \in [\frac{1}{m}] \tag{2.4}$$

The good example calibration in Section 2.2 achieves an ASCE value of zero, which means the model is calibrated. The bad calibrated example on the other hand achieved an ASCE value of 0.5. This is not the worst possible, which is due the fact that bin 50% is calibrated.

2.3.3 GASCE

Some approaches will not only use bins for calibration, but also groups. These groups are used to achieve calibraiton in subsets of the data. To measure calibration in specific groups it is necessary to have a function $g(x)$ that assigns a given sample to a group g. A group defines a subset of the samples. For example a possible group could be a certain region for a weather forecast. It could be possible that given the location the predicted probabilities are calibrated differently. The concept of groups will be explained with more detail in the Concept Chapter 3. The group average squared calibration error (GASCE) equals the ASCE. The only difference here is that score is calculated for every possible group.

$$GASCE(g) = \sum_{b=1}^{n} P_{b,g} \times (E[y - f(x)|(x,y) \in S_{b,g}])^2, \text{ where } b \in [\frac{1}{m}] \text{ and } g \in G \tag{2.5}$$

For this score no exemplary value will be provided because the examples from the previous Section 2.2 had no groups yet. Mainly it works analog to the ASCE. A higher value means worse calibration and a value near zero means that the specific group is better calibrated.

2.3.4 MSE

The mean squared error (MSE) is also known as the brier score [1]. This score is mainly used to measure accuracy. Detommaso et al. [6] state that the MSE is not a direct measure of calibration. It is shown that the MSE can be broken down into the ASCE and the variance of the model. Though it is possible that the model with lower MSE can still be poorly calibrated.

Equation 2.6 shows how the MSE is calculated. X is the set of samples, Y is the set of labels and n is the count of samples. For every sample the difference between the label and the confidence of the

model is calculated. The label can either be 0 or 1, which stands for correct (1) and incorrect (0). The sum is then normalized with the number of samples n.

$$MSE = \frac{1}{n}\sum_{k=1}^{n}(y_n - f(x)_n)^2, \text{where } y \in Y \text{ and } x \in X \tag{2.6}$$

To give some insights into the workings of the MSE here are the values for the good and bad calibration example from Section 2.2. The well calibrated example achieves an MSE of 0.125. The bad calibrated example gives a MSE of 0.625. Therefore lower MSE values can be an indicator for better calibration.

2.3.5 Skill score

The final score that is used to measure the calibration of a given model is the skill score. To specify the skill score it is necessary to introduce the MSE reference score which is defined by Equation 2.7 [16]. To calculate the MSE reference score every sample is put into one big bin. In this bin the probability that a sample is correct $P_{correct}$ is calculated. This is also called a naive estimator that predicts that every sample is correct with the probability $P_{correct}$.

$$MSE_{ref} = P_{correct} * (1 - P_{correct}) \tag{2.7}$$

The skill score equation is displayed in Equation 2.8. Essentially the skill score determines the improvement over the naive estimator baseline MSE_{ref} [16]. A negative skill score means that the calibration is worse than the baseline MSE_{ref}. Is the skill score positive it can be a sign of better calibration. A skill score of 0.05 and above is seen as good forecast quality by the Deutsche Wetterdienst [32].

$$\text{skill score} = \frac{MSE_{ref} - MSE}{MSE_{ref}} \tag{2.8}$$

For the good calibration example from Section 2.2 the MSE_{ref} has a value of 0.25 and a skill score of 0.5. This means the calibration is better than the calibration of the naive estimator. The bad example on the other hand has the same MSE_{ref} of 0.25, but a skill score of -1.5 indicates worse calibration than the naive estimator.

2.4 Calibration Methods

This section aims to outline the calibration methods that are used in this work. The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning, were used by Detommaso et al. [6] on the subject area of question answering with LLMs. In this work these methods are used to calibrate generated code. Each method is described briefly at this point. In the later work these calibration methods will be explained in more detail.

Histogram binning The histogram binning approach sorts a training set's probabilities in bins. Then the correctness per bin is calculated. This is done by checking how many samples are correct in each bin. After this the difference between the correctness and average confidence are calculated. This difference is called the bin's „delta“. The deltas are calculated for each bin and then used to correct the

samples in each bin with the corresponding delta. This delta is then added to the actual probability of a sample. This is a simple calibration approach.

Linear regression The linear regression approach is the first approach that uses the concept of groups. A group is a categorie that describes a sample. A sample can be in multiple groups. The linear regression learns a bias and weights for each group. These bias and weights are then used to predict the delta. Like in histogram binning the delta is then added to the actual probability of a sample. Through this the approach learns the connection of groups and the needed corrections to achieve better calibration.

Iterative group histogram binning The iterative group histogram binning approach combines the concept of groups with bins. It can be seen as an extension of the histogram binning approach. The approach iteratively improves the calibration of selected bin-group combination. This bin-group combination is selected by calculating the most „promising "bin-group combination and the highest delta value. The approach stops if the maximal GASCE multiplied by the probability that a sample is in the given group is smaller than the α value. The hyperparameter α is predefined and determines the bin size.

Iterative group linear binning The last approach is an improvement of the iterative group linear binning approach. Therefore three improvements are made. With the first improvement the changes are not made on a single bin-group combination but rather on cumulative bins. This means the algorithm is less overfitting and comes to a halt faster, because improvements impact more samples. The second improvement helps the algorithm to stop overfitting. Therefore early stopping mechanisms are implemented. Every iteration the approach checks if the MSE is still decreasing on a validation set. Furthermore another adjustable parameter ϵ is introduced. This parameter stops the algorithm if the subset that should be adjusted is too small. With the last scaling a linear scaling is applied in the cumulative bins to adjust the confidence values. This linear scaling learns an alpha and beta value by minimizing the MSE on the probabilities in the cumulative bins.

2.5 Related Work

This section presents different works that were published in the years before, which have a connection to the subject of calibration and LLM generated code.

A survey from Xie et al. [23] gives an overview of confidence estimation and calibration methods for black box LLM. These black box LLMs only allow an interaction via the API of the provider. To estimate the confidence of the generations of such LLM three main categories of criteria are discussed. Consistency, Self-Reflection and Hybrid methods. The consistency methods have on the one hand the similarity based approach. This approach evaluates the similarity between the multiple responses for the same prompt and uses these metrics to derive a confidence score. On the other hand is the entropy-based method. This approach calculates an uncertainty quantification with the help of graph Laplacian eigenvalues or semantic set quantity. Another method is the self-reflection approach. This approach is a verbalization method in which the prompt is designed to give the model the opportunity to output its confidence in its generated output. The hybrid methods combine both approaches. Furthermore post processing calibration methods such as histogram binning and isotonic regression are discussed. Besides that there are also approaches that try to train an auxiliary model to achieve better calibration.

The work of Spiess et al. [16] deals with the calibration and correctness of code LLMs. It is shown that these LLMs can be helpful, but can generate faulty code. The tested LLMs (CodeGen2, Codex, GPT-3.5) are not well calibrated out of the box. This means that the confidence of the LLMs does not correlate well with the probability of correctness of the generated results. Furthermore it is shown how to improve the calibration with Platt scaling, which helps to better classify the results.

The work from Ni et al. [11] deals with the ability of LLMs to generate code. They found that the models which perform best are not the ones that are best calibrated. Better models are according to Ni et al. [11] more reliable, because the calibrated confidence can be used as a good indicator to evaluate the generated code.

Virk et al. [20] investigate the calibration of confidence score for natural language code summaries. A confidence score which mirrors the similarity of the a code summary from a human is created. To calibrate their results they used Platt scaling. Virk et al. [20] found that their results provide reliable results for the predictions.

Vaithilingam et al. [18] conducted a user study with software developers. These developers were equipped with the GitHub Copilot assisted to solve programming tasks. They found out that the GitHub Copilot generated correct code in most cases. But a significant time saving could not be discovered. This is due to the fact that the developers still need to check the code for correctness and bugs are found the developer first needs fully understand the generated code to fix the problem. Nevertheless the generated code can be helpful to serve as a starting point.

The work from Zhong and Wang [25] suggests a new kind of dataset to evaluate LLM generated code. This dataset enables the evaluation of the reliability and robustness of LLM generated code which interacts with APIs. It is argued that previous datasets and benchmarks are restricted to small programming tasks, which does not reflect the everyday programming tasks. Even GPT-4 achieves a misuse of 62% of API interactions. This can lead to huge damage in real world applications.

Chapter 3

Concept

Our goal is to create a model f that gets a raw uncalibrated probability and input features for a code generation and outputs a well calibrated probability. That means that a model f, which estimates a better calibrated probability, is created. This probability is defined by $P(y = 1|x)$, where $x \in X$. X is the set which contains all our code generation samples and Y is the set of labels. If the sample is correct $y = 1$ or incorrect $y = 0$. Such a well calibrated probability helps the software developer to better interpret the code generations and serves as a indicator for possible bugs.

In Diagram 3.1 all main components that are used in our pipeline are shown. The left side shows the code and data generation and the right side displays the calibration part. The process starts with the code generation prompt. This prompt is put into an LLM to generate a program. Besides the generated code also the metadata like the token scores is fetched. This data is processed to get a raw uncalibrated probability for each generated sample. This serves as a baseline to compare our calibrated probabilities against. After the generation, the samples are evaluated for correctness. This is done by using test cases. After the evaluation, groups are formed. This groups are used to summarize samples which have the same properties. For example one group could contain each sample that is written in the programming language python. The concept of groups is used in some of our calibration approaches. Their purpose is to only correct samples that are in a certain group. Therefore better calibration is achieved in this groups.

Furthermore, an optional process which involves another LLM is explored. This LLM is used to evaluate the generated code. Therefore a LLM is selected and input the generated code with an instruction to evaluate that generated code. This outputs either a percentage of correctness or a qualitative description of the generated code. This LLM evaluation output can either be treated directly as calibrated scores or be used for further calibration. After these step the outcoming data is used for calibration.

After the preparation phase the main part of our work begins where several calibration methods can be chosen. These methods are histogram binning (HB), linear regression (LR), iterative group histogram binning (IGHB) or iterative group linear binning (IGLB). It is measured how well the calibrated scores align with the true probability of a code sample to be correct before each method is started and after the method was used. Besides using every method on their own its also possible to use every method at once. This is used to compare the outputs of each method.

The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning are used from the paper multicalibration for confidence scoring in LLMs from

Detommaso et al. [6]

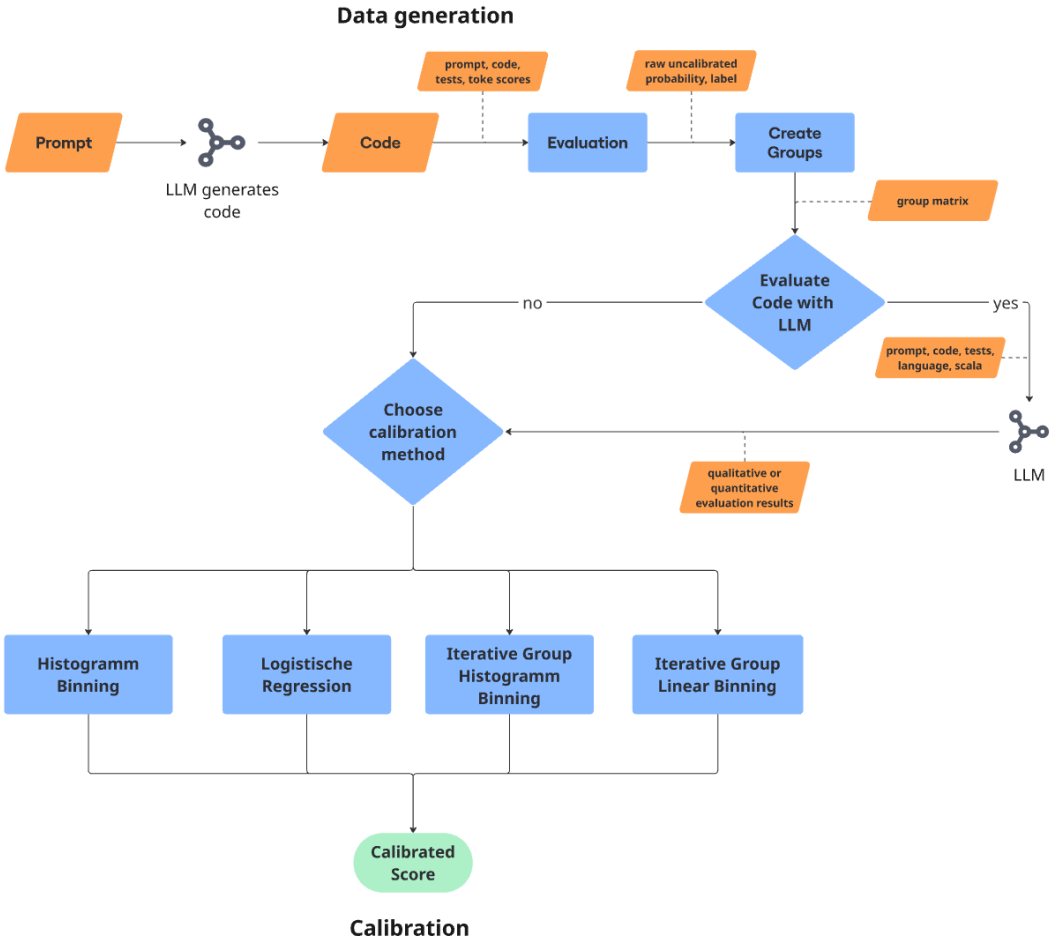


Figure 3.1: Calibration process pipeline. Our process is separated into two main parts. The data generation process and the calibration process. During the data generation all necessary data for our calibration is gathered. Which data is gathered is displayed in the orange boxes. For the calibration part one of the four approaches can be chosen or all approaches are used for comparison.

3.1 Data generation

At the data generation beginning stands a prompt. This prompt specifies the criteria for the code to be generated. Depending on the dataset, this prompt consists of imports, a method hull and a doc string. The method hull contains the function name, input parameters with their data type and the expected output data type. The doc string contains a description that specifies the functionality which is expected of the code. In most cases the prompt also contains examples. The examples specify cases fo which inputs which outputs are expected. In Listing 3.1 a example prompt from the humaneval dataset is shown. This prompt is written in python.

```python
from typing import List

def sort_numbers(numbers: str) -> str:
    """ Input is a space-delimited string of numberals from 'zero' to '
        nine'.
    Valid choices are 'zero', 'one', 'two', 'three', 'four', 'five', 'six',
         'seven', 'eight' and 'nine'.
    Return the string with numbers sorted from smallest to largest
    >>> sort_numbers('three one five')
    'one three five'
    """
```

Listing 3.1: Sample prompt from the humaneval python task 19. The prompt contains an import, the definition of the function with expected output type and a docstring with a description of the task. Furthermore the prompt got an example which shows the model the expected return for a given input. [4]

The prompt is forwarded to a LLM. The LLM then tokenizes, embeds the prompt and predicts the method body. This is done with the selected decoding strategy.

For the generation vllm [7] is used. Vllm is a fast and easy to use framework to conduct inferences with LLMs. Specific LLMs can be selected through vllm's so called model. This can be any model from huggingface hub or a downloaded model. For the generation the prompts are passed into the generate() method from vllm. This method also takes sampling parameters. These specify parameters like the temperature, number of generations, stop words, max tokens and logprobs. Which parameters are used during the experiment will be specified in the evaluation Chapter 4.

The generated token sequences $t_1, ..., t_n$ are returned in the response from the LLM. These sequences contain the completed code pieces. Besides these sequence additional details about each token are fetched. This data contains an entry for each generated token. This entry contains the token id, the actual log probability and the actual decoded token. The token probability of the i-th token is defined by $P(T_i = t_i | t_1, ..., t_{i-1})$. Furthermore, the cumulated token probability, which sums up all token probabilities, is fetched. Besides the information about the output tokens information about the input tokens is gathered.

In this work open source models are used. These model allow easy access to token probabilities. These token probabilities indicate how confident the model is in the selection of the token. Note that the returned probabilities are log probabilities and need to be converted into normal probabilities to allow an intuitive assessment. In Figure 3.2 a correct generation for the prompt from Listing 3.1 can be seen. The generation is cut of at the first not intended comment, which is located at the last row in Figure 3.2. The colors indicate the confidence of the model in the token: a high confidence is colored dark green and a low confidence is colored in red.

Figure 3.2: The Figure shows the generated code for humaneval task 19 3.1. Each token is highlighted with a color. This color indicates confidence of the model in the specific token. A red color means high uncertainty and a green color means high certainty from the model in the specific token.

The above generation process is applied to every sample in the given dataset, obtaining one generation for each prompt. To summarize, each sample has a prompt, the generated completed code, and details about the token sequence most importantly the individual token scores.

Evaluation

After the generation process the correctness of the generated code completions is checked. This is done by applying test cases that are provided with each sample by the respective research dataset. The test cases check if the generated code produces the correct output for a given set of input values. To do so the prompt with is concatenated with the generated code completion and the test cases are appended. Resulting in a program that wen can execute. Depending on the result of the execution it is specified if the sample is correct or incorrect: if it is executable and passes all test cases the sample gets the label correct ($y = 1$). This label is defined as y. Fails the sample test cases or is not even executable it is labeled as incorrect ($y = 0$). Note at this point that test cases can have blind spots and may not cover every possible input-output combination.

To summarize, the data contains for every sample the program that consists of the prompt, code completion and the tests, the raw uncalibrated probability and the label which indicates if the sample is correct.

Groups

For some calibration methods that are discussed later features are needed that describe the sample. These features are called groups. A group is a subset of the dataset. These groups can also be intersecting. For example a piece of generated code is written in the language python and has examples in the initial prompt. It could be assumed that LLMs have a worse structural correctness and therefore calibration with different kind of programs. Maybe they have better calibration on samples which contain examples and a worse calibration on sample with long input prompts. That would be intuitively the case for humans.

The groups are associated with the samples through a matrix. Let N be the number of samples and M be the number of groups. Then the matrix is N x M. This matrix contains zeros and ones which display the affiliation of the sample and the groups. Each row is a sample and each column represents a group. A zero means that the sample does not belong to this group and a one means it does. This matrix is built during the data loading process and will be used in different calibration approaches in the next sections. A sample matrix is displayed in Figure 3.3. The columns G_m stands for the groups and the row X_n stands for the samples. The set G contains all groups. And the function $g_m(x) = 1$ specifies if the given sample $x \in X$ is in the given group G_m.

$$\begin{array}{c} \\ X_1 \\ X_2 \\ \\ X_n \end{array} \begin{array}{c} \begin{array}{cccc} G_1 & G_2 & \dots & G_m \end{array} \\ \begin{bmatrix} 0 & 1 & \cdots & 1 \\ 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 0 \end{bmatrix} \end{array}$$

Figure 3.3: The group matrix has the size N x M, where N is the number of samples and M is the number of groups. The group function $g_m(x)$ assigns the sample x to a group G_m.

In this work three different approaches to create groups are used:

- The first grouping style is named „simple“. This grouping style is our the entry point into the group topic. Simple characteristics are used to build the groups matrix. The checked criteria are: the prompt contains the word "example", is the prompt longer than the median prompt, and is the program code longer than the median program code. The idea behind the example group is that it should intuitively be easier to create a program if you have examples that show how the code works. The same goes for the group that checks the prompt length. It is assumed that the problem becomes more complex the longer the prompt is. For the last group, which checks the length of the generated program, there is an assumption that longer generated code is more likely to be incorrect than shorter code.
- Our second grouping style approach is called „complexity“. For this the generated programs are analyzed. This is done with command line tool called „scc“which was created by Ben Boyter [28]. This tool outputs the lines of code, lines of comments and an approximation of cyclomatic complexity per program. The cyclomatic complexity is a metric that measures the independent paths through a program and was introduced by Thomas McCabe [10]. It should be noted that the complexity score is only an approximation because its difficult to automate the calculation over multiple different program languages because of their differences in programming paradigms [28]. A group for different complexity levels as defined by Thomas McCabe [29] is created. McCabe [29] introduces a measure of complexity levels which determine risk. The first group contains all samples that have a cyclomatic complexity of 1-10. These

samples are considered to have a small risk. The second group contains all samples that have a complexity of 11-20 and have moderate risk. The third group contains all samples with a complexity of 21-50 and have a high risk. The last group contains all samples that have >50 complexity and have a very high risk. These four groups contain all samples and are therefore self-contained.

- The third grouping style places the samples into groups which represent semantic categories of programming problems. These categories are extracted from the task definition that is contained in the prompt. To achieve this, the prompt is checked for certain keywords and then added to the group. The first group is build to represent tasks that use mathematical operations and numbers. This assignment takes place through the check for keywords like 'integer', 'float' and a combination of 'number' and 'calculate'. The second group represents all samples that operate with strings or chars. Therefore keywords like 'string', 'char' or 'word' are being searched. The last group contains all samples that use higher level data objects like lists. Here, keywords like 'list', 'array' or 'tree' are checked for.

3.2 Evaluation through a LLM

One straightforward approach comes to mind to estimate our desired probability $P(y = 1|program)$: simply another another LLM could be asked if the generated code is correct. Such auto-evaluation is quite common in question, answering, for example `https://autoevaluator.langchain.com/`.

Therefore a method is used to evaluate the generated code with the help of a LLM. The LLM can either be the same LLM or another one can be chosen. This is helpful to evaluate the generated code with a even more powerful LLM. To get an evaluation of the generated programs, a prompt is built which contains the programming language, the program, which contains prompt, generated code and test cases, and an instruction to evaluate the correctness of the code. The actual prompt templates can be found in the appendix B.1. The prompts and approaches are inspired by the work of Ulmer et al. [17]. Note that the evaluator LLM does not need to be a open source model and it is also not necessary to retrieve the log probabilities. This is possible because the generated token sequences are directly used from the evaluator LLM and no token log probabilities are used.

For the evaluation of the generated code there is the differentiation between the quantitative and the qualitative approach.

- The quantitative approach tries to directly get a percentage of correctness from the model. Therefore the model is asked to return a percentage of correctness from the range of 0 to 100. This percentage of correctness is then treated as the confidence in the correctness of the sample. It is assumed that it is difficult for a LLM to rate the program by using a continuous number range.
- The qualitative approach, on the other hand, gives the model the option to choose from a preset of qualitative descriptions. For example „High“or „Very low“. These qualitative scale can be found in Table 3.1. Based on the selection the model makes the sample gets assigned a specific percentage. These values are defined independently and can also be adjusted after the model made its prediction.

For both methods template matching is used to extract the values from the answer token sequences. For the qualitative approach a number with a percentage symbol is searched for and for the qualitative

Description	Probability
Very low	0
Low	0.15
Somewhat low	0.30
Medium	0.45
Somewhat high	0.60
High	0.75
Very high	0.90

Table 3.1: Qualitative scale mapping which is used to rate the correctness of the generation with the evaluator LLM. If the evaluator LLM returns the description low the sample gets a probability of 30%.

approach the predefined qualitative descriptions are searched. The first occurrence is used for that purpose.

The probability outputs of these method can than be used for further calibration methods in the process. It is also possible to not use any calibration method and analyze the pure outputs from the evaluator LLM.

3.3 Calibration foundations

For our calibration approaches a probability is needed which reflects the confidence in the generated code. The first baseline is obtained by using the the confidence from the individual token probabilities. It is assumed that the model is „insecure“about its generations these token score will be low / have a low entropy. With the individual token probabilities the average token log probability per sample is calculated. This is done by the accumulation of the token log probabilities. This number is then divided by the number of generated tokens. The result is then transformed into actual probabilities by using the exponential function. The formula is displayed in 3.1. It is assumed that these average probabilities are the confidence of the model in the generated code for further experiments. Lower average token probabilities imply an uncertainty in the model output. Higher confidence scores imply that the model thinks that its output is more reliable. As a results of this process step, an probability is calculated for each sample that represents the confidence.

$$P^{raw}(x) = exp(\frac{\sum_{k=1}^{n} logprob_k}{n}) \tag{3.1}$$

These uncalibrated probabilities of the code samples are the baseline for the further work. They are used to verify if a better calibration with the calibration methods can be achieved. These calibration methods are displayed in the following sections.

3.3.1 Discretizing scores

To review the calibration on the generated data, the probability range is splitted into bins. This is done by splitting the range into equally large bins. They are defined by $[\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, ..., \frac{m-1}{m}, 1\}$. The number of bins is controlled with the parameter m. To assign this probabilities to a bin a assignment has to be made that decides which probability belongs to which bin. To achieve this the distance to

each bin is calculated and the bin which is closest to the probability is assigned. The Equation to calculate said assigned bin can be seen in 3.2. $f(x)$ can be the baseline probabilities of the generated code or the adjusted probabilities from a calibration approach. For the baseline $f(x)$ would be $P^{raw}(x)$. $f'(x)$ represents the discretized values.

$$f'(x) = argmin(|f(x) - b|), \text{ where } b \in [\frac{1}{m}] \text{ and } x \in X \tag{3.2}$$

This mapping is then used to calculate the correctness in each bin b. This is done before and after the calibration method was applied. Through that the calibration on the uncalibrated data, the calibration on the calibrated data and the increase or decrease of metrics that were showed in Chapter 2 can be evaluated.

3.3.2 Visualizing calibration result

To compare the approaches metrics are calculated on the uncalibrated data and then after each method was used. This makes it possible to get an idea of how well each method performed under the same circumstances. Reliability charts and histograms are used to plot the changes that each calibration method achieves. The reliability chart shows each bin and the correctness of the samples in per bin. The histograms show the distribution of the samples in the grid and can therefore track where the distribution mass moved. In Figure 3.4 an example for a reliability chart can be seen. On the x-Axis confidence values that are received from the LLM are displayed. On the y-Axis the probability of correctness for the bin is displayed. The blue diagonal line symbolizes a perfect calibration. The bar colors of the reliability diagram display the sample distribution. Darker blue shades mean more samples are in this bin and light blue or white displays less samples. Furthermore the numbers about the bars show the actual count of samples in this bin.

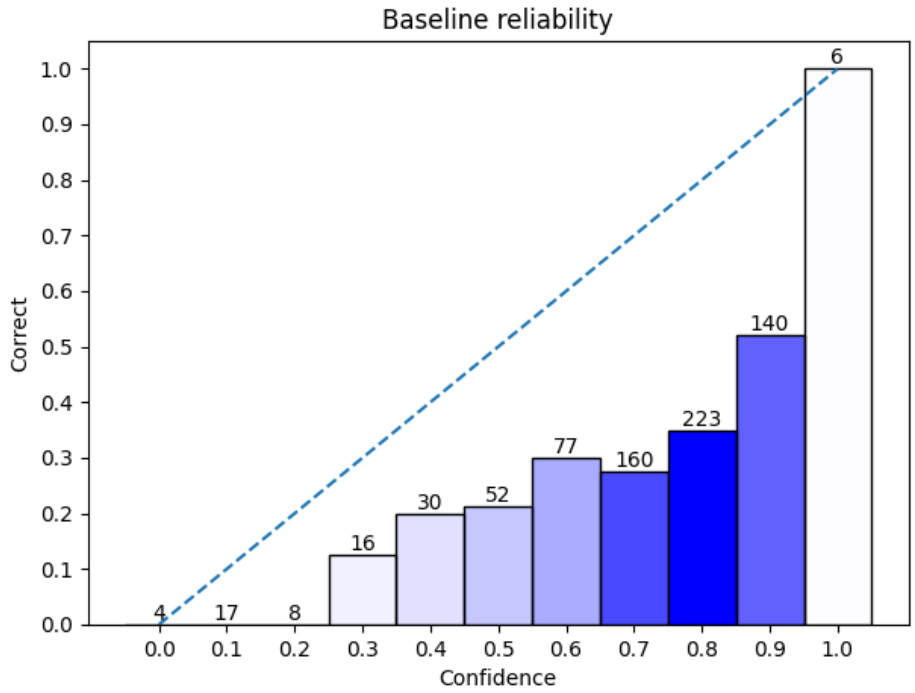


Figure 3.4: Reliability chart to display calibration. The x-axis displays the confidence of the LLM and the y-axis shows the correctness in each bin. Above each bar are the number of samples in the bin. The dotted line indicates perfect calibration. The color gradient shows where the most samples are located. Darker blue means more samples.

In Figure 3.4 the uncalibrated data distribution of a dataset can be seen. The probabilities are unevenly distributed and are not close to the diagonal line which would be the perfect calibration of the underlying data. Perfectly calibrated data would mean that in bin 0.5 are always 50% correct and 50% incorrect samples. The bin with a confidence 0.8 contains 223 samples. It is the bin with the most samples an there for has the darkest shade of blue. The raw uncalibrated probabilities express a high confidence (80%). But the samples in this bin have a correctness of around 35%. This is an example of poor calibration. For a good calibration the bar should be at 80%.

3.4 Calibration

A calibration method is something that gets the raw uncalibrated probabilities and turns them into better calibrated probabilities. These better calibrated probabilities should provide a better estimation of the correctness of a generated code sample. Some of these methods do not only take the raw uncalibrated probabilities, but also use groups. These groups are used to obtain a better calibration on the subset of samples that are contained in this groups. The assumptions is that better calibration can be achieved by choosing a good combination of groups.

3.4.1 Histogram binning

The first calibration approach is called histogram binning (HB). This approach is the simplest of those studied in this thesis and tries to adjust all probabilities in a bin at once. To achieve the calibration improvement, the difference between the correctness and the average confidence in each bin is calculated. These so called deltas in formula 3.3 are learned under supervision on a trainings set. Because the values in each bin are discretized the average confidence is always the value of the bin. For example in the bin 0.5 the average confidence is 0.5. The correctness is calculated by using the label of the samples. With the these two values the difference (delta) between optimal calibration and actual calibration is calculated. This is done for every bin.

$$delta_b = E[y - f'(x)|(x, y) \in S_b], \text{ where } b \in [\frac{1}{m}] \tag{3.3}$$

The calculated deltas are then used to update all discretized probabilities in each bin. These updates are made to every defined bin. This is done by adding the corresponding delta with the discretized probabilities in the selected bin. The result of this are new adjusted probabilities for every sample. This process shifts the probabilities to a better calibrated version. But it neglect the fact that different bins and the contained samples are not always disjoint and do maybe depend on each other [6].

3.4.2 Linear regression

The second method that is used to calibrate the raw uncalibrated probabilities is linear regression (LR). In this approach the groups appear. The groups are used as input features for the linear regression. In Formula 3.4 these groups are displayed as x_i and can take the value one or zero. As output the difference between the label y and the actual probability $P^{raw}(x)$ which is displayed as $delta$ is wanted. The label y defines if a sample is correct or not and therefore can take the values zero or one. Which means incorrect sample will be shifted into the direction of zero because they become a negative sign and the correct sample will be shifted more to the direction of one because of the positive sign.

$$delta = b + w_1 * x_1 + w_2 * x_2 + ... + w_n * x_n \tag{3.4}$$

To realize this the linear regression learns the weights w_n for each group and the bias b. In this approach the actual probabilities are used and not the discretized values. Further more the bias b of the linear regression is used to adjust samples that have no groups associated with. These weights and the bias are learned on a training set.

With the learned weights $w_1, ..., w_n$ and bias b the adjustment for a given group feature input $x_1, ..., x_n$ is predicted. The predicted y value is then added to the raw uncalibrated probability of the sample. $f^{LR}(x) = y + f^{raw}(x)$. Through that a better calibrated probability for the sample is achieved.

3.4.3 Iterative group histogram binning

Iterative group histogram binning (IGHB) is a approach which combines the idea of groups with the concept of bins. Instead of calculating deltas for each bin, like it was the case in the histogram binning 3.4.1 approach, these deltas are calculated for every set of bin-group combination. These sets are defined by: $S_{b,g}$, where $b \in [\frac{1}{m}]$, $g \in G$. The deltas are computed by calculating the difference between the correctness Y and the average confidence $f'(x)$ for every set of bin-group combination as in Formula 3.5. Through this a matrix of delta values is created. Where M is the number of bins and G

is the number of groups. The matrix has the shape of M x G.

$$delta_{b,g} = E[y - f'(x)|(x,y) \in S_{b,g}], \text{where } b \in [\frac{1}{m}], g \in G \tag{3.5}$$

The problem that arises is, that multiple delta values for each sample exist. That is the case because a sample can be in multiple groups, but only in one bin. So instead of applying the delta to every sample in a bin, like the histogram binning approach did, one bin-group combination is selected. To calculate which bin-group combination to adjust, the deltas from Equation 3.5 are squared and multiplied with the probability $P(S_{b,g})$, where $b \in [\frac{1}{m}]$ and $g \in G$. This probability tells how the samples are distributed over the bin-group combinations. For the formula see 3.6. By doing this the bin-group combination is chosen which has the highest impact on the calibration of the the data.

$$(b,g) = argmax(delta^2_{b,g} * P(S_{b,g}), \text{where } b \in [\frac{1}{m}] \text{ and } g \in G \tag{3.6}$$

The adjustments of the sample probabilities are made in a loop. This loop stops if the maximal error in the data is smaller then a predefined value α. This α value also defines the grid size in this approach. The α value of 0.1 results in a grid of 10 bins. So the grid is defined by calculating $\frac{1}{\alpha}$ and using this value as the bin width. The max error on the other hand is recalculated on each iteration of the loop. It is defined as the result of multiplying the GASCE with the probability of which a sample is in a given group as in formula 3.7.

$$\text{max error} = argmax(GASCE * P(g)), \text{where } g \in G \tag{3.7}$$

In each iteration the probabilities are adjusted on all subsets (train, valid and test set) of our data. For evaluation purposes the changes which each iteration made are tracked. After each iteration the calibrator is fitted with the adjusted probabilities. That means each deltas ar calculated again and it is checked if the error is still greater than the α value. Is that not the case the adjustment stops and the calibration process is finished.

Through the specific adjustment that are made for the bin-group combinations there is a high risk of overfitting on the data. That is caused by the small amount of data that is adjusted in each iteration.

To apply the learned changes onto an new sample a history of every adjustment that is made per iteration must be saved. Therefore the bin-group combination that was adjusted and the delta which was applied needs to be saved. It is important to mention that these changes need to be applied in the exact same order as they where saved. That is because through the changes of time samples are moved into other bins and could be affect by the next change in another way.

3.4.4 Iterative group linear binning

Iterative group linear binning (IGLB) improves the previous IGHB approach. Therefore three adjustments are made. First the corrections are not applied for exactly one bin-group combination, but to cumulative bins instead of individual ones. A bin is now defined like $X <= b2$ not $b1 <= X <= b2$, where b_n is the edge of a bin. This set is defined by the parameter $\otimes$. $\otimes = \leq$ all values that are smaller than or equal the selected bin are adjusted. Analog a $\otimes = \geq$ means all values that are greater than or equal the selected bin are adjusted. The parameter $\otimes$ specifies this cumulative distribution function. This means our sets $S_{b,g}$ from the iterative group histogram binning are now extended with the parameter $\otimes$. These new sets are defined as follows: $S_{\otimes,b,g}$, where $b \in [\frac{1}{m}], g \in G$, $\otimes \in \{\leq, \geq\}$. The update of a larger number of samples is assumed to help to limit the risk of overfitting.

Secondly the so called linear scaling is applied within each bin. This scaling is defined by the formula 3.8:

$$f^{LS}(x) = expit(alpha + beta * logit(f(x)) \tag{3.8}$$

$f(x)$ are the probabilities of the samples and $f^{LS}(x)$ are the new adjusted probabilities. To get an optimal set of alpha and beta values the MSE 2.6 on Formula 3.8 is minimized. This is done by using the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm [21]. As initial start parameter 0 is used for alpha and 1 is used for beta. By applying the formula the alpha and beta values are calculated for all sets $S_{\otimes,b,g}$. In the prediction phase the set which is has to be adjusted is selected. Equation 3.9 shows how the adjustments are only made on a specific subset $S_{\otimes,b,g}$.

$$f^{IGLB}(x) = \begin{cases} f^{LS}(x), & x \in S_{\otimes,b,g} \\ f_t(x) & \text{else} \end{cases} \tag{3.9}$$

To choose the subset, that needs correction, the probability that a sample is in a given subset with parameter $\otimes$, group and bin ($P(S_{\otimes,b,g})$) is calculated. The delta values are also calculated like the iterative histogram binning approach did. The only difference is that the delta values are not calculated on exact bins, but on the cumulative ones. The probability $P(S_{\otimes,b,g})$ is then multiplied with the deltas squared. The set $S_{\otimes,b,g}$ with the largest potential to improve calibration is selected. This calculation is formalized in the formulas 3.10 and 3.11. Algorithm 1 shows the process of the iterative group linear binning for better understanding.

$$delta_{\otimes,b,g} = E[y - f'(x) | (x,y) \in S_{\otimes,b,g}], \text{ where } b \in [\frac{1}{m}], g \in G\ , \otimes \in \{\leq, \geq\} \tag{3.10}$$

$$(\otimes, b, g) = argmax(delta^2_{\otimes,b,g} * P(S_{\otimes,b,g})), \text{ where } b \in [\frac{1}{m}], g \in G \text{ and } \otimes \in \{\leq, \geq\} \tag{3.11}$$

The third improvement specifies conditions that lead to an early stopping of the algorithm. The early stopping is also to prevent overfitting. After Detommaso et al. [6] adjustments on too small subset could impact the generalization ability of the approach. Therefore the parameter ϵ is used for early stopping. If ϵ is greater then the probability $P(S_{\otimes,b,g})$ of the chosen subset the algorithm will terminate. The second criterion checks if the MSE is still decreasing on the validation data set. If it stops decreasing or stays the same the algorithm will come to an halt.

To apply the changes onto a new sample, that has not been learned, all changes that where made during the iteration onto the sample have to be applied. The changes need to be in order the where done while training.

Algorithm 1: Iterative group linear binning

Data: $D_{train}, D_{test}, D_{val}$
ϵ
$t \leftarrow 0$
$f_t \leftarrow f'$
Result: f_t
while True **do**
  $mse_t \leftarrow MSE_{D_{val}}(f_t)$;
  $(\otimes, b, g) = argmax_{\otimes,b,g}(delta^2(f_t) \times P(S(f_t)))$ with $b \in [\frac{1}{m}], g \in G$ and $\otimes \in \{\leq, \geq\}$;
  if $P(S_{\otimes,b,g}) < \epsilon$ **then**
    break
  end
  $f_{t+1} \leftarrow f^{IGLB}_{\otimes,b,g}$;
  $mse_{t+1} \leftarrow MSE_{D_{val}}(f_{t+1})$;
  if $mse_{t+1} \geq mse_t$ **then**
    break
  end
  $f_t \leftarrow f_{t+1}$
end

3.4.5 Regressor

calibration approaches are combined as features in a unified rgressor

In our last approach different regressors are compared. With these regressors the correction terms are predicted which are used to update the probabilities like in the approach 3.4.2. These delta values are displayed in Equation 3.12. x represents all the features that describe the code sample. ($y_{1:m}$ is the generated code from the LLM). $f_\ominus$ represents the regressor that is used.

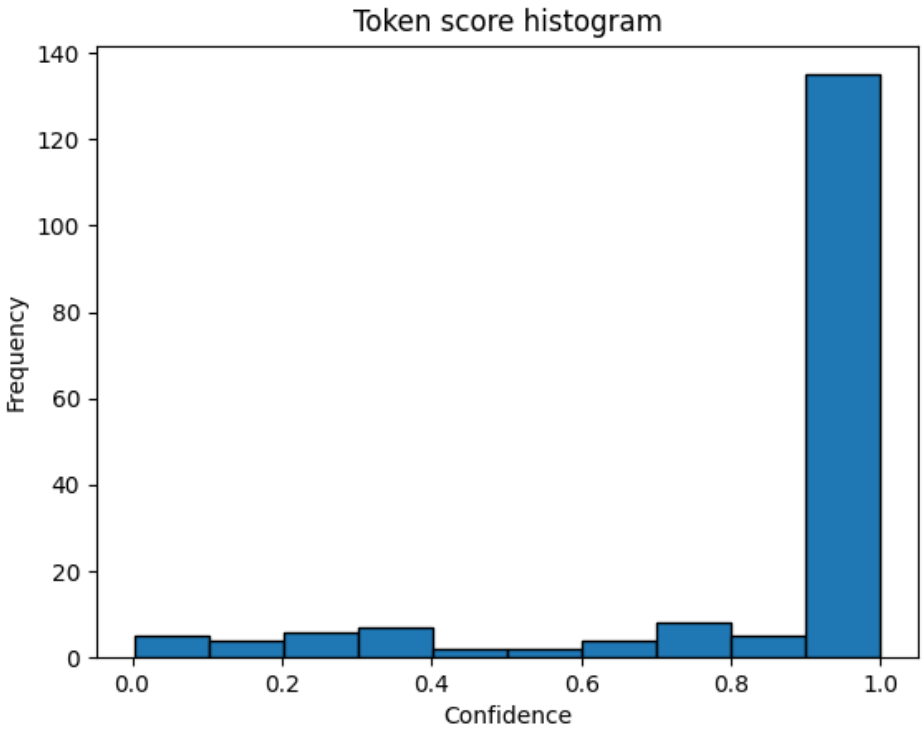
$$delta(y_{1:m}) = f_\ominus(x) \tag{3.12}$$

The new input features consists of:

x

- The groups that are assigned to the sample. The definition of these groups did not change and are used like described in Section 3.1. Wieviele Merkmale?
- The assigned sample probability. Here the average token probability is used. (1 value?)
- A histogram was created by gathering the individual token log probabilities. These were transformed into normal probabilities and summarized using a histogram. The histogram bins are divided into ten bins. A sample histogram can be seen in Figure 3.5. The assumption is that the distribution of token scores correlates with the correctness of a sample. Is the distribution of the histogram more to the probability of 100% than it is expected that the sample is more likely correct. Is the distribution shifted to the probability of 0% than the sample is more likely to be incorrect.

War die Einschätzung des (Judge) LLMs nicht auch noch ein Feature?



A

Figure 3.5: Histogram that displays the distribution of the token confidence in the humaneval task 19. The confidence is displayed on the x-axis and the number tokens in each bin are on the y-axis. It can be seen that the model is mostly overconfident in its token selection.

This (10 values...) serves as an input feature for...

These

This input features where used to train the regressor. As target value the deltas to adjust the probabilities for better calibration are predicted. The regressors where trained on a test subset of the data.

d.h., was genau sind die labels auf denen trainiert wurde??

The following regressors where used:

Was sind jeweils die Parameter theta, die trainiert werden?

- Linear regression.
- Support vector regression (SVR) [13, 3]
- XGBoost [5]

The linear regression is used as described in Section 3.4.2. The difference here is the extended input feature.

(ansonsten Standard-Parameter?)

For the SVR the radial basis function is used and for XGBoost the default settings are used to fit our data.

All three regressors get the same input features and same target value. For the prediction process the output value of the regressor is used to adjust the sample probability. The assumptions is that the regressors learns deltas that improve the calibration of our data. In this approach no bin assignments are needed for the calibration process. That is because the approach tries to learn the adjustments with the given input feature constelation.

d.h. am Ende ist die Vorhersage : \hat{y} := \hat{y}(uncalibrated) + f_theta(....)?

Chapter 4

Evaluation

Our goal is to evaluate if the calibration approaches introduced in Section 3 show an improvement in calibration, i.e. yield more reliable estimates of the correctness probability. Furthermore it is shown which approach provides the best results. Besides that the datasets and benchmarks are displayed that are used in this work. After this the used models are displayed and insight into the used parameters in the generation process is provided. In the following section the results for each approach is displayed. This is done visually and with the metrics from Chapter 2. Experiments which look into some aspects of our calibration approaches are also conducted. Such as different initial confidence predictions with the integration of an evaluator LLM. This was mentioned in the Section 3.2. Furthermore the calibration with different grouping styles is investigated. These experiments are shown in Section **??**.

4.1 Data

As data for the experiments the humaneval dataset [4] is mainly used. This data consists of 164 programming tasks in python. Each sample consists of a prompt, which describes the programming task. The prompt contains imports, a method hull and a doc string. An examples prompt is given in Listing 3.1. The tasks are build to assess language comprehension, reasoning, algorithms and mathematic [4].

This 164 samples are too few for our calibration approaches, this will be shown in on of our Experiments 4.5. Therefore the MultiPL-E benchmark [2] is chosen. The benchmark makes it possible to translate the humaneval tasks into 22 other programming languages. Through this a sample count of 3661 is achieved. The benchmark uses a compiler to translate the test cases, types, doctests and the python terminology. So the evaluation for correctness works like for the python tasks with unit test cases. The samples are evaluated in a docker container. This is because executing generated code can be potentially dangerous. The generations are cut off at a certain stop word. For python this is for example the word „\ndef“which would mark the start of a new high level function.

Generation works over a command line tool where several parameters are specified. These parameters are the model, the dataset (either humaneval or MBPP), the programming language, temperature, batch size, completion limit and our output directory. By default these benchmarks do not return the token log probabilities which are mentioned in the Section 3.1. To return them some changes are made to the existing MultiPL-E scripts. The logprob parameter is added in the automodel_vllm script and the results files are adjusted to contain the token ids, log probabilities and the cumulative log

probabilities. One generation is generated for each sample and batch processing is used to generate multiple programs at once.

The calibration approaches are trained on 60% of our data. The other 40% are used for validation (20%) and testing (20%). Cross validation is only used in certain cases because the data is sufficiently large enough. The validation subset is necessary for the IGLB calibration approach, where it is used for early stopping.

The output of each method is stored in a folder structure. This folder structure is created while loading the data for the calibration approach. It changes dynamically when the approach is called with another parameter set over the command line. The following is a example how the structure looks: `runs/humaneval-all-keep-Qwen2.5\_Coder\_7B-Instruct-1.0-comp-1/lr/linear/avg\_logprob/categories/split` The runs folder contains all results that are generated. The second folder is the run which describes the dataset and for code generation also the model with which the samples where generated. After this the calibration approach is specified in this case its „lr“and under this folder the different binning types can be found. For this work only linear binning is considered so its the only possible option. The next option specifies which method is used to generate the probability which is associated with the sample. The options are the average token probability (avg_logprob), the quantitative or the qualitative probabilities which where generated by the evaluation LLM. After this a folder exists which specifies the grouping style which were explained in the section 3.1. The last folder contains either the results for the partitioned data or for the run with all data and no split into train, test and validation.

In the deepest folder the charts which display the calibration before and after the method was used and a table which contains the calculated metrics are found.

4.2 Models and parameters

The model is loaded through vllm and is then used to create the code generations. As model the Qwen2.5-Coder-7B-Instruct is used. At the time of this work this is the best performing 7B-Model on the BigCodeBench Leaderboard [27]. BigCodeBench is a benchmark that involves solving tasks with code. It focuses on different functions and complicated instructions.

The model was loaded from the huggingface hub. For the temperature the value of 1 is chosen and kept the default value for top-p as 0.95, which means randomized generations.

4.3 Results

In this section firstly the calibration of our average token probability baseline is displayed. Then the results of the different calibration approaches are shown. Therefore the improvement of calibration compared to our baseline is displayed. The baseline uses the average token log probabilities per sample. To create our grid the parameter $m = 20$ is used. This creates a grid of 21 bins. That means all our probabilities get discretized to values in $0, 0.05, 0.1, ..., 0.95, 1.0$. This grid will be used to calculate the calibration metrics and diagrams for all used approaches. This enables us to compare the approaches later on. The parameter $m = 20$ was chosen by doing a grid search over multiple values for m.

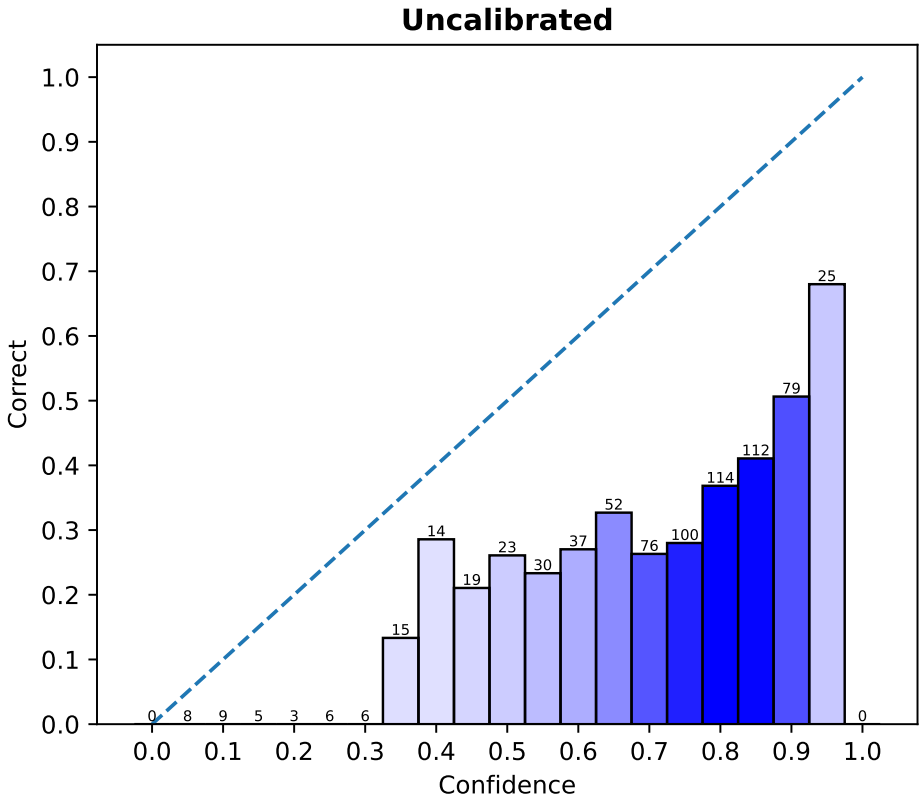


Figure 4.1: Reliability chart that displays the baseline calibration. On the x-axis the confidence of the model in the generated code is displayed and on the y-axis the correctness in each bin is shown. The majority of the samples (326 samples) are located between the confidence of 75%-85%.

In Figure 4.1 the baseline for our following experiments can be seen. The most samples are located around the bins with a confidence of 75%-85%. The height of bin 80% indicates that only 40% of the samples are correct. This means that the LLM overestimates the correctness of the code generations. Only the bin with a confidence of 0% has perfect calibration. Every other bin is not well calibrated. For example the 50% bin has only a correctness of around 25%. Which means these samples should be in the bin with a confidence of 25%. In general the bars are below the diagonal line. This means that the correctness of each bin is lower than the actual confidence. Therefore the model systematically overestimates the correctness of the generated code.

ECE ↓	ASCE ↓	MSE ↓	MSE_{ref}	Skill Score ↑
0.376	0.151	0.355	0.222	-0.599

Table 4.1: Baseline metrics of the uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Table 4.1 shows the metrics for our baseline. The ECE, ASCE and MSE scores are high. The lower

these scores are, the better is the calibration. A value of zero for the ECE and ASCE would mean that the model is perfectly calibrated. The MSE_{ref} is a measure of a naive estimator [15]. This estimator places all samples into one bin an measures the correctness in this bin. This value is then used as the confidence for all samples. This estimator is defined by $MSE_{ref} = p_c * (1 - p_c)$. p_c is the probability of correctness over all our samples. The value of 0.222 means that roughly $\frac{1}{3}$ of our samples are correct. The MSE_{ref} will be used to compare our approaches and is part of the skill score. The skill score measures the improvement of the respective calibration via the MSE relative to the MSE_{ref} score. If its positive this indicates an improvement of calibration if it is negative it is a indicator for poor calibration. The baseline is not well calibrated an therefore needs adjustments.

Group	GASCE ↓
> Median Prompt Len	0.090
not > Median Prompt Len	0.065
examples	0.038
not examples	0.117
> median code	0.089
not > median code	0.065

Table 4.2: GASCE of the baseline with uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Furthermore the values for the GASCE are shown in Table 4.2. The GASCE makes it possible to measure the calibration error for each group independently. For our baseline the „simple“grouping style is used. In the later experiments the effect of other groups is demonstrated. The Table 4.2 shows each group with there name or descriptive feature and the corresponding GASCE value. The greatest misscalibration is in the group „**not** examples“. And the best calibrated group is the „examples“group. This means that samples with an example are better calibrated out of the box. These leads to the hypothesis that the examples help the LLM to better asses its confidence.

4.3.1 Histogram binning

To recapitulate, the histogram binning method used deltas which correct every sample in a bin. This deltas represent the difference between the correctness and the confidence in the given bin. The same parameter $m = 20$ is used to create the grid like in the baseline. This will be done for all the following approaches. The deltas for each bin where learned on the training subset and applied to the samples in the test subset. Groups are not mentioned, because they do not play a role in this calibration approach. Never the less it is possible to indirectly improve the calibration of a group through improving the calibration on the whole dataset.

In Figure 4.2 the results of application of the approach are shown. Left column of the Figure 4.2 represents the test subset before calibration and the right column represents the data after the histogram binning was applied. This kind of diagram will be used for every approach that is shown in this results section. The distribution of the samples shifted from high confidence values to much lower confidence value. For example before the histogram binning was used the bin 80% contained the most samples. Afterwards the most samples are located in the bin with the confidence value of 45%. Furthermore the data is better calibrated and lines up better with the dotted line, which represents perfect calibration. Nevertheless it did not worked for the test data on all bins as can be seen on the

bin with 95% confidence. This bin has a correctness of 70%. But the samples where shifted into the bin with 50% confidence. So the approach learned a delta value that does not generalizes well. This could be due the small amount of samples in this bin.

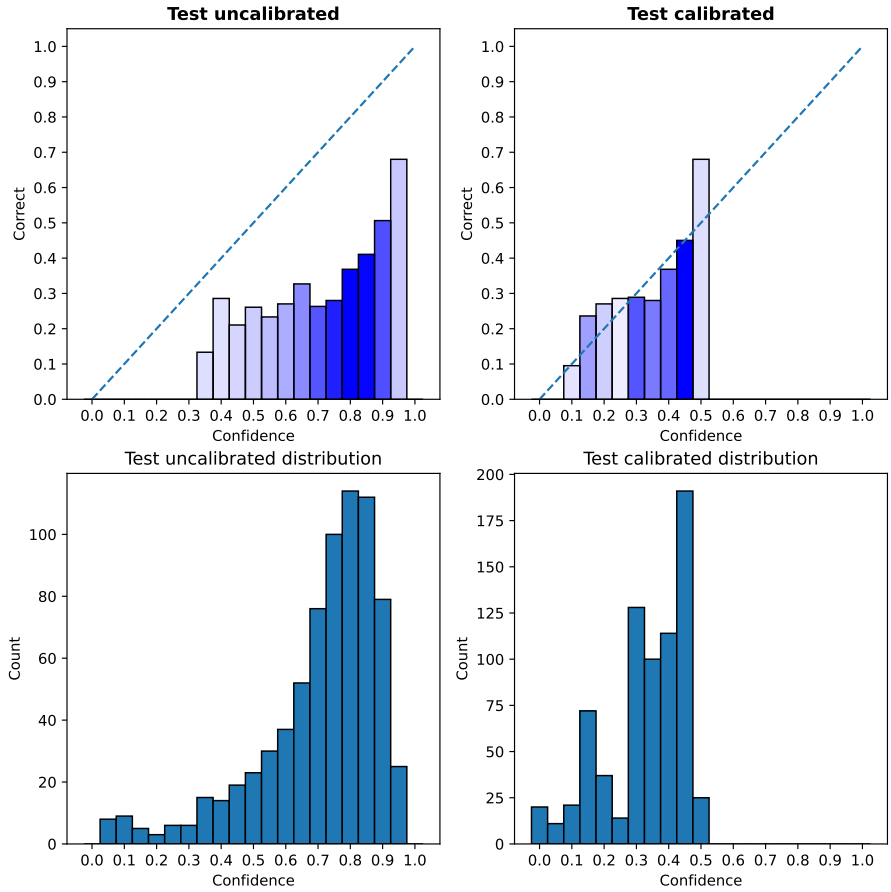


Figure 4.2: Displays the calibration before (left side) and after (right side) the histogram binning approach was used. On the x-axis the confidence of the model in the generated code is displayed. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution of our samples is displayed.

That the approach improved the calibration can also be seen in the metrics that are tracked. They are shown in Table 4.3. The ECE decreased by -0.340 which is a good improvement and ASCE improved to a value close to zero. This assures us that the results are better calibrated then before. Furthermore the MSE decreased by -0.146. Which is not as much as expected. But Nevertheless the skill score improved by 0.658 and is now positive which indicates a better calibration and an improvement over the naive estimator MSE_{ref}.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.036	0.003	0.209	0.222	0.059
Difference	-0.340	-0.148	-0.146	0	0.658

Table 4.3: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To sum up the learned deltas in the histogram binning approach managed to generalize to the test data. It also achieved a better calibration compared to our baseline with the average token probabilities. Although it does not achieve perfect calibration and still has weakness for bins with low sample count.

4.3.2 Linear regression

Our next method is the linear regression. The goal with this approach is to learn the weights w_n for the defined groups G and a bias b to predict the delta. This delta is then used to correct the raw uncalibrated probability. Note that the corrections are made on the raw uncalibrated probabilities and not the discretized values. This is the first approach which uses the groups as a feature to determine the correction needed. Therefore the metrics are shown in Table 4.4 and the GASCE is shown in Table 4.5. This shows how the approach improved the calibration in each group.

Figure 4.3 shows the visualized change of calibration. Again the left side shows the uncalibrated test data and the right side shows the data after the linear regression was used. The sample distribution shifted to the left. Generally the calibration of each bin improved but on some bins the approach did not generalized well, at least for the view of all samples. In the comparison results the calibration for each group individually is shown. Generally the calibration was improved through this approach but also did not achieve perfect calibration.

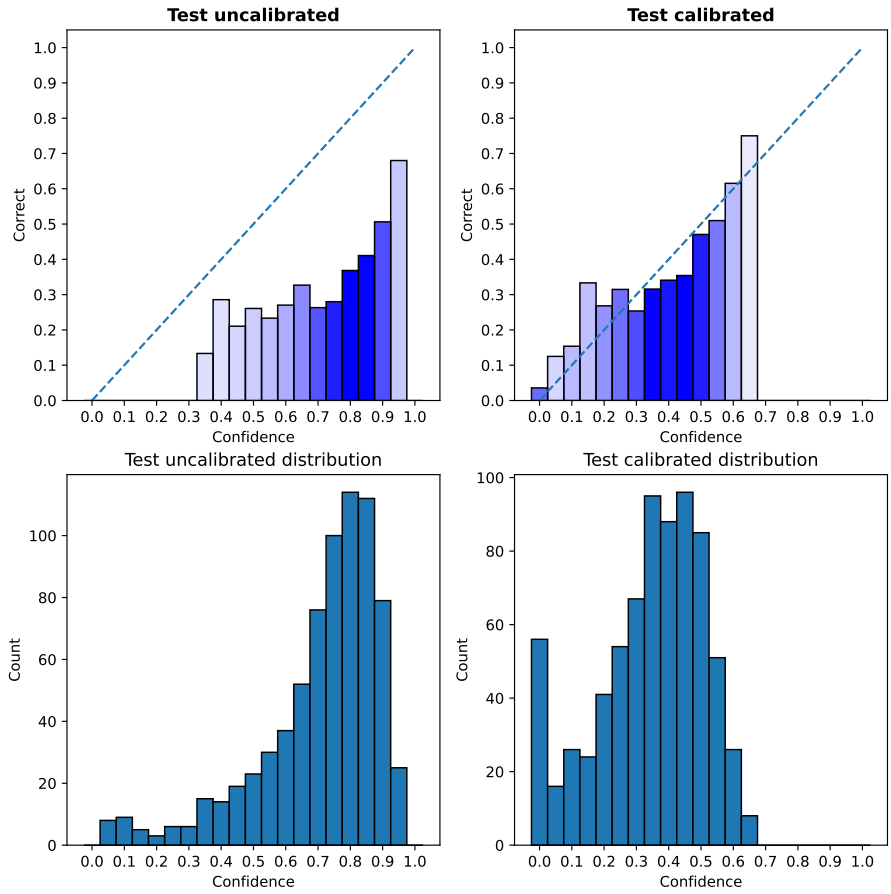


Figure 4.3: Displays the calibration before (left side) and after (right side) the linear regression approach was used. On the x-axis the confidence of the model in the generated code is displayed. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

The metrics in Table 4.4 show that despite of the visualization the calibration improved quite much on the test dataset. The calibration errors ECE and ASCE improved significantly and also the MSE decreased by -0.148. The skill score turned positive which indicates good calibration. Furthermore the GASCE is displayed for every group in the Table 4.5. Every group achieved an improvement in calibration and only has an error in the three-digit decimal range close to zero. This means our groups are well calibrated.

	ECE ↓	**ASCE** ↓	**MSE** ↓	**MSE$_{ref}$**	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.057	0.004	0.207	0.222	0.068
Difference	-0.319	-0.147	-0.148	0	0.667
HB	0.036	0.003	0.209	0.222	0.059

Table 4.4: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.090	0.001
not > Median Prompt Len	0.065	0.004
examples	0.038	0.001
not examples	0.117	0.004
> median code	0.089	0.002
not > median code	0.065	0.004

Table 4.5: The table shows the GASCE for each group. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To conclude the linear regression approach achieved a good calibration on the overall test dataset. With problems to adjust some samples. Reasons for this could be that the groups are not well chosen. However this opposed by the fact that the GASCE, as in Table 4.5, improved on every group.

4.3.3 Iterative group histogram binning

The iterative group histogram binning used bin-group combinations to make adjustments to the raw uncalibrated probabilities. To use this approach the parameter α is needed which is determined by our parameter m and our stopping condition. For this method the value $m = 20$ is chosen which means the α-value is 0.05. These value was selected after doing a grid search. This means 21 bins are used and the algorithm comes to a halt when the max error is smaller then 0.05.

Figure 4.4 shows that the distribution of our samples shifted again to the left side. This means the actual probabilities where to high and through the calibration process the confidence values in the correctness of our samples was lowered. The calibration improved because the most samples are located around the bins 35% and 40% which are reasonably good calibrated. The other bins are not well calibrated. A reason for this could be that the algorithm did not targeted the samples in those bins, because the error is to low through the low sample count in those bins.

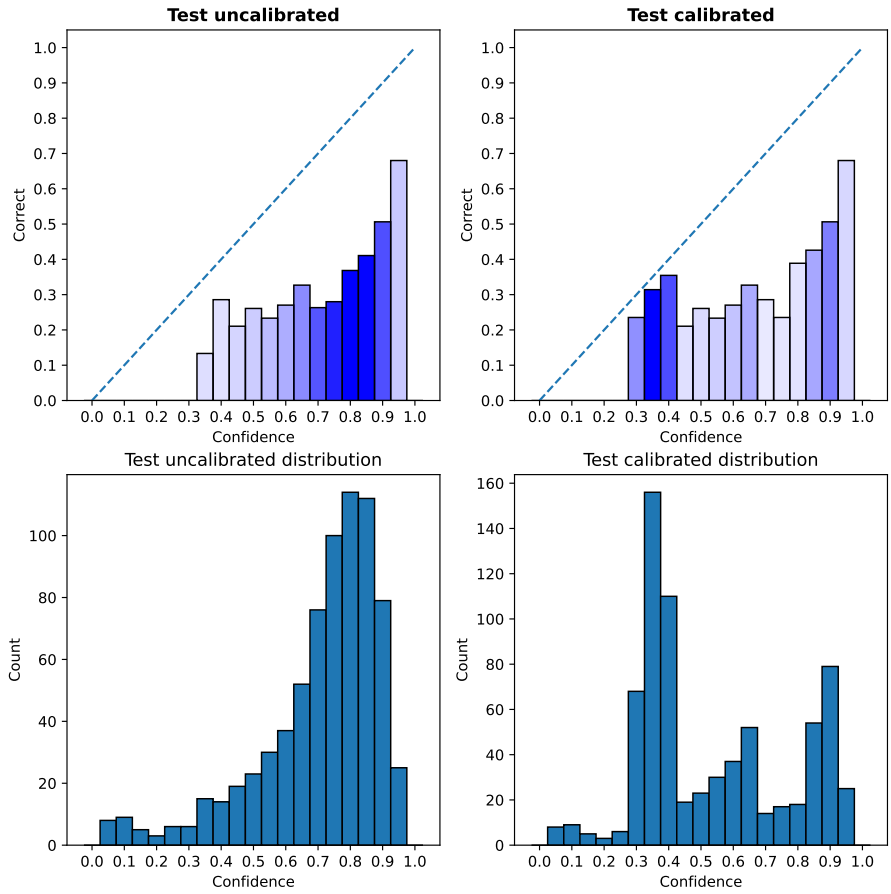


Figure 4.4: Displays the calibration before (left side) and after (right side) the iterative group histogram binning approach was used. On the x-axis the confidence of the model in the generated code is displayed. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

For further evaluation of the iterative group histogram the metrics are shown in Table 4.6 and Table 4.7. The ECE achieved an improvement of 0.171. Which means that our bins are have a lower misscalibration then before. But the value is still high. The ASCE and MSE also improved. The skill score does not reach a positive value. This indicates that the calibration is worse then the naive estimator baseline M_{ref}. Furthermore Table 4.7 shows that the GASCE decreased on all groups. Especially the „**not** examples“groups achieved a way better calibration.

	ECE ↓	**ASCE** ↓	**MSE** ↓	**MSE$_{ref}$**	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.205	0.068	0.275	0.222	-0.239
Difference	-0.171	-0.083	-0.080	0	0.36
HB	0.036	0.003	0.209	0.222	0.059

Table 4.6: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.09	0.042
not > Median Prompt Len	0.065	0.030
examples	0.038	0.033
not examples	0.117	0.039
> median code	0.089	0.037
not > median code	0.065	0.034

Table 4.7: The table shows the GASCE for each group that is used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

The conclusion is made that the approach slightly improves calibration, but has the tendency to overfit on the trainings data. Because of this tendency the approach was tested with cross-validation and 5-Folds. The results showed that the generalization is poor on all folds.

4.3.4 Iterative linear histogram binning

Our last approach is the iterative linear histogram binning. This is a improvement of the previous iterative group histogram binning approach. The main improvements are the adjustment on cumulative bins, the linear scaling in said groups and the early stopping with the MSE on the validation data set and the parameter ϵ. For ϵ the value of 0.01 is chosen. ϵ is the parameter that stops the algorithm from making adjustments for too small subsets and therefore should prevent overfitting. Note that the bin width is again determined by the value m.

The visualized results of the approach can be seen in Figure 4.5. Before the approach was used the most samples have a confidence in the range of 75%-85% and no bin is well calibrated. After the approach was used the most samples have a confidence between 30%-45%. The confidence values were greatly reduced but are now more in line with the correctness value of these bins. The bins with less samples 55%-95% are not well calibrated after the calibration process.

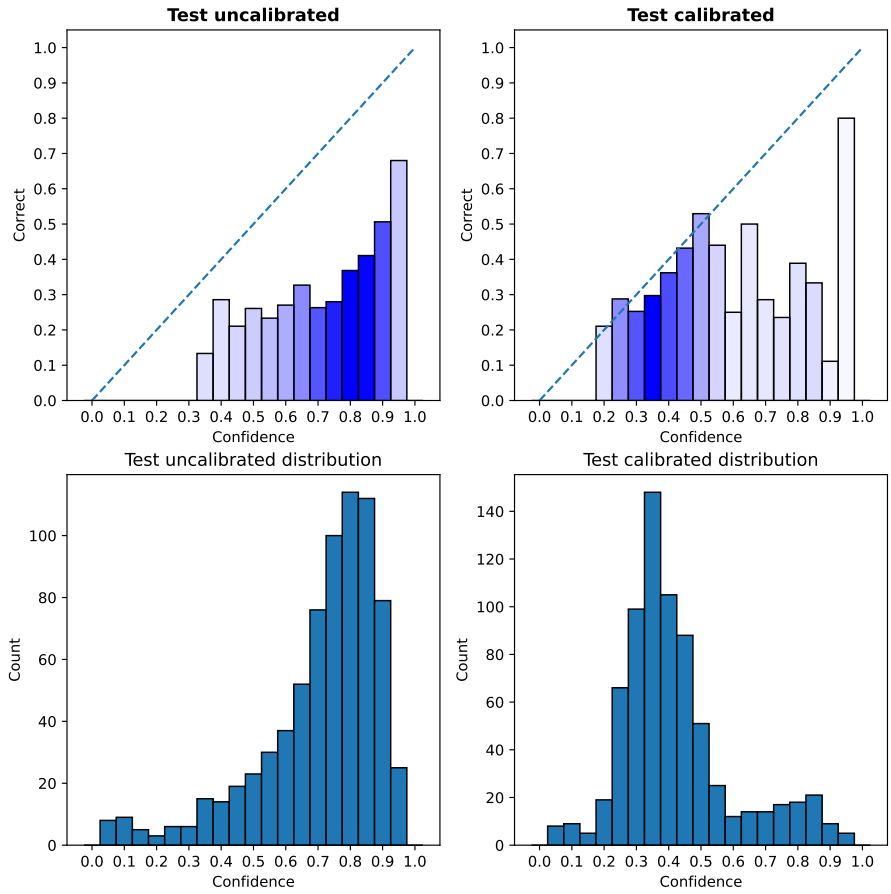


Figure 4.5: Displays the calibration before (left side) and after (right side) the iterative group linear binning approach was used. On the x-axis the confidence of the model in the generated code is displayed. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

The metrics in Table 4.8 show that the calibration has improved. The ECE decreased by 0.275, the ASCE by 0.118 and the MSE by 0.112. The skill score measure shows also an improvement against our raw uncalibrated probability baseline, but it does not achieve a positive value. The GASCE, which can be seen in 4.9 improved on almost all groups. Only the „examples“group has the same value as before. A reason for this could be the small amount of samples (396) in this group. Thereby the max error is to small an the group is not targeted. However the best improvement achieved the „**not** examples“group, which has a GASCE of nearly zero.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.101	0.033	0.243	0.222	-0.095
Difference	-0.275	-0.118	-0.112	0	0.504
HB	0.036	0.003	0.209	0.222	0.059

Table 4.8: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.09	0.022
not > Median Prompt Len	0.065	0.017
examples	0.038	0.038
not examples	0.117	0.003
> median code	0.089	0.023
not > median code	0.065	0.016

Table 4.9: The table shows the GASCE for each used group. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To summarize the iterative group linear binning approach achieved a better calibration against our baseline. Nevertheless it should be mentioned that the strong improvement of only one group is an indicator that the algorithm came quickly to a halt. Reasons for that could be the stagnating MSE on the validation set or an improper ϵ value.

4.3.5 Comparison

To summarize a comparison of the approaches is made and it is checked which one achieved the best calibration results. Therefore they are first compared visually with each of the reliability diagrams as in the sections before. This can be seen in Figure 4.6. Then the calibration which is achieved inside each group is shown and the metrics for each method are checked.

Figure 4.6 shows the reliability diagrams for each method. Confidence on the x-axis and correctness on the y-axis. The chart in the first row displays the baseline calibration on the test subset. The second row contains the histogram binning and linear regression calibration charts. The last row the iterative group histogram binning and iterative group linear binning results.

Visually the histogram binning and linear regression approaches show the best calibration results. The iterative group histogram binning approach on the other hand looks slightly better calibrated, but worse then the other approaches. The improved version, the iterative group linear binning, shows again better results then the iterative group histogram binning approach but worse then histogram binning and linear regression.

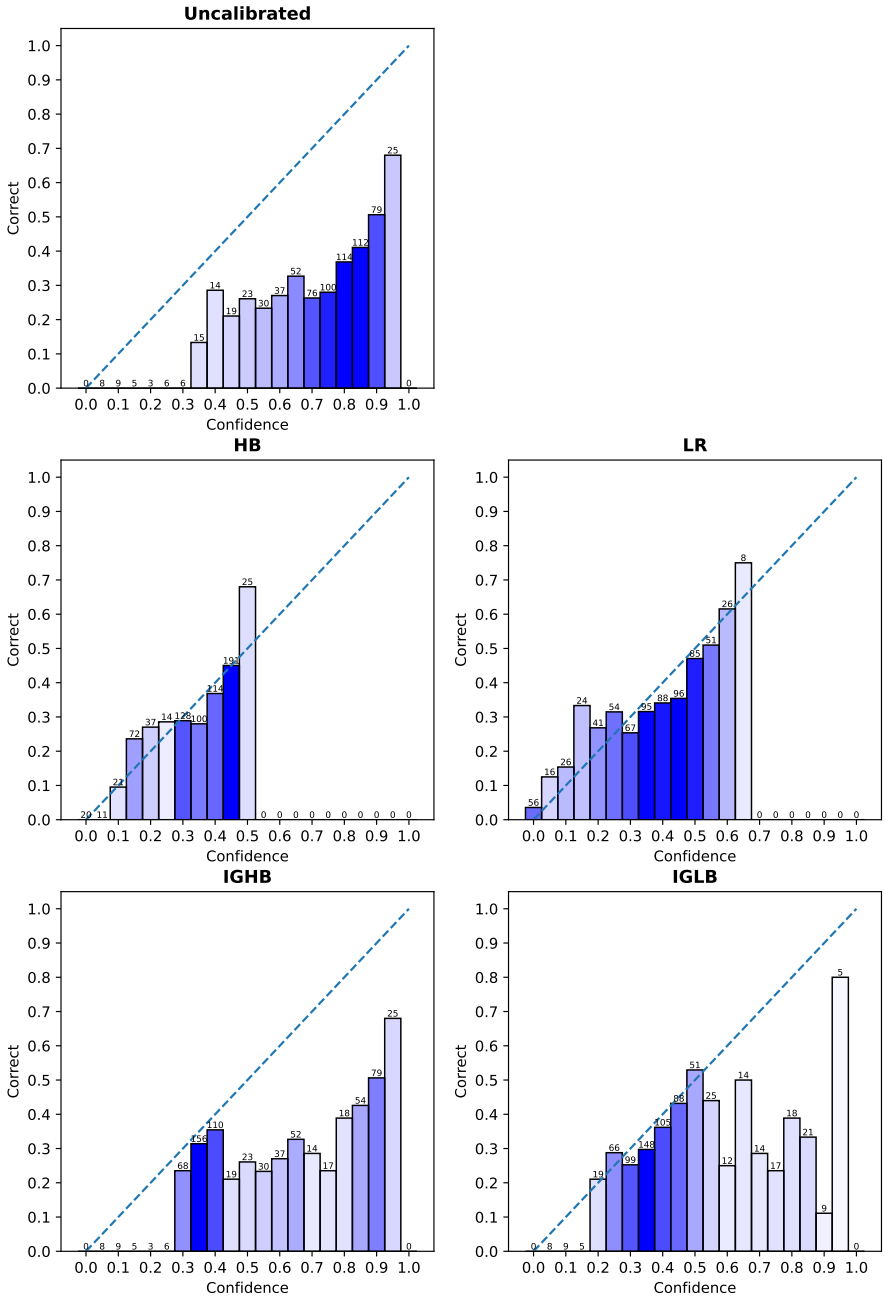


Figure 4.6: Displays the calibration before and after each approach was used. On the x-axis the confidence of the model in the generated code is displayed. The y-axis displays the correctness in each bin. Over each bar the samples count is displayed. The first reliability chart shows the uncalibrated data. The other charts stand for each approach that is used.

What can be seen visually is also reflected in the metrics, which are displayed in Table 4.10. The uncalibrated row stands for the baseline with average token probabilities. All other rows are the corresponding approaches that where described in previous sections. The simple histogram binning approach works the best on the metrics with lowest ECE and ASCE. Not far off comes the linear regression which was the first approach that used groups to achieve a better calibration. Linear regression achieves the lowest MSE and therefore the best skill score. The worst performance has the iterative group histogram binning with highest ECE, ASCE, MSE scores and lowest skill score. That is the case of the poor generalization ability and overfitting on the training dataset. The improvements that where introduced with the iterative group linear binning have enhanced the metrics, which can be seen in the last row. Nevertheless the improvements were not enough to beat the histogram binning or linear regression approach.

Method	**ECE** ↓	**ASCE** ↓	**MSE** ↓	**MSE$_{ref}$**	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
HB	**0.036**	**0.003**	0.209	0.222	0.059
LR	0.057	0.004	**0.207**	0.222	**0.068**
IGHB	0.205	0.068	0.275	0.222	-0.239
IGLB	0.101	0.033	0.243	0.222	-0.095

Table 4.10: Shows the metrics for each approach that are used for comparison. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓				
	Uncalibrated	HB	LR	IGHB	IGLB
> Median Prompt Len	0.09	**0.001**	**0.001**	0.042	0.022
not > Median Prompt Len	0.065	0.005	**0.004**	0.030	0.017
examples	0.038	0.003	**0.001**	0.033	0.038
not examples	0.117	**0.003**	0.004	0.039	**0.003**
> median code	0.089	**0.002**	**0.002**	0.037	0.023
not > median code	0.065	**0.003**	0.004	0.034	0.016

Table 4.11: The table shows the GASCE for each used group. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

The next Figure 4.7 is structured the same as Figure 4.6. The charts show the calibration per group. Each circle represents one group. The circle diameter indicates the number of samples in the group. The assignment is as follows:

- > Median Prompt Len ●
- **not** > Median Prompt Len ●
- examples ●
- **not** examples ●
- > median code ●
- **not** > median code ●

On our baseline every group has an approximated confidence of 70%. These confidence values are the average confidences per group. Every groups is far off from being well calibrated. In the diagram for the histogram binning approach the groups where just shifted to a lower confidence value. This makes sense because groups were not used in this approach. The linear regression on the other hand achieves a solid calibration for nearly every group. The yellow and blue group are slightly off the perfect calibration line. Furthermore the confidence shifted from 70% to a range of 25%-40%. Iterative group histogram binning betters the calibration on nearly every group except the green group which has less samples then the other ones. It is expected that the group with low sample count has a worse calibration, because the error in this group will be lower. That is because the probability of each group plays a role in calculating said error. The improved iterative group linear binning approach achieved better calibration than its predecessor, but also does not succeed in correcting the green group. It is noted that the probability distribution in each group can vary and could contain information of the group calibration.

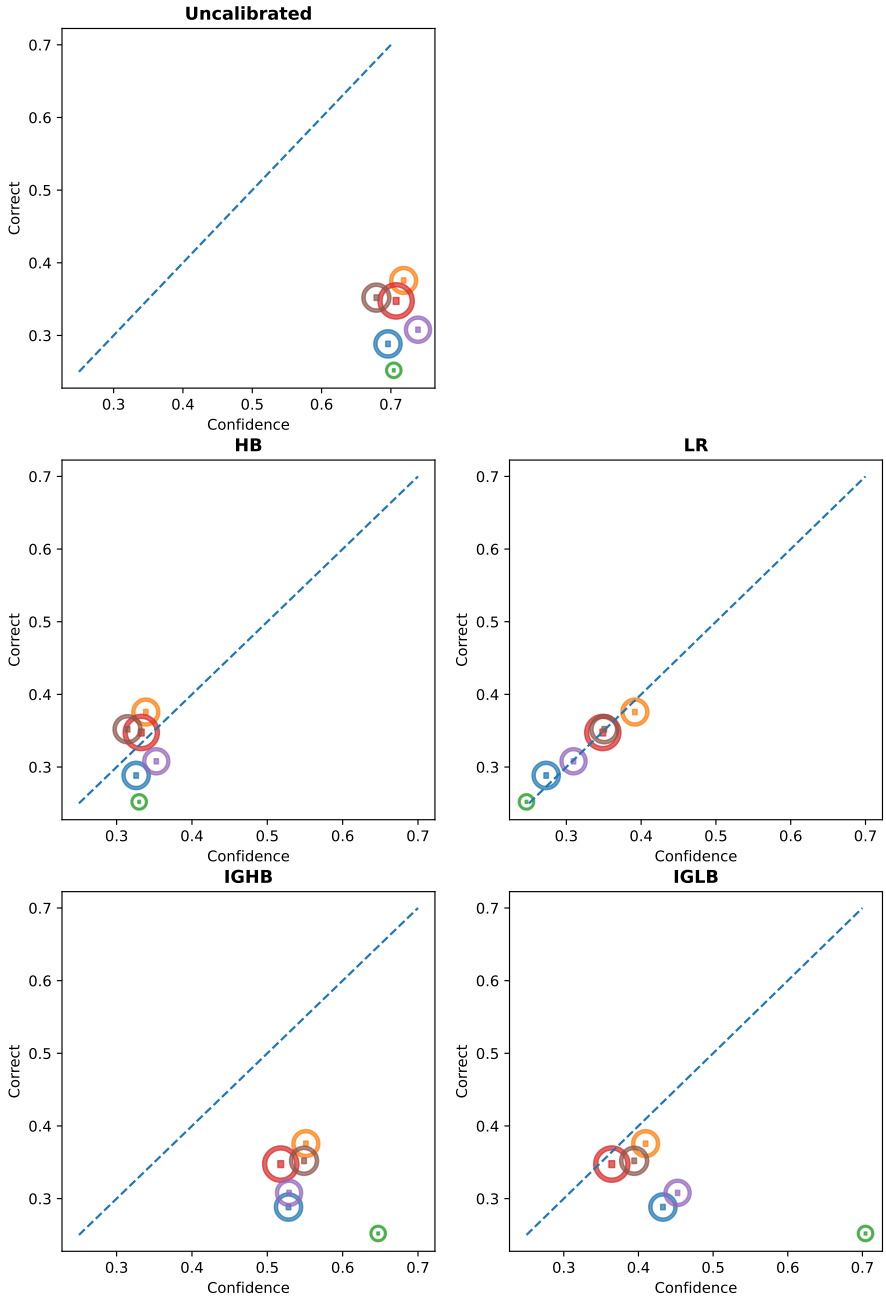


Figure 4.7: Group calibration for the uncalibrated data and then for each approach that is used for calibration. Each circle stand for a group. The size of the circles indicate the amount of samples. Confidence is on the x-axis and the correctness of the group is in the y-axis.

In conclusion: the simple approaches histogram binning and linear regression delivered the best calibration results. And the iterative group histogram binning and iterative group linear binning approach performed worse but not quite bad. Furthermore the linear regression delivered the best calibration for the groups.

4.4 Evaluator LLM

Now that overview of the results of the average token probability baseline was presented, two more ways to get an initial probability of correctness for a sample are explored. Therefore two approaches that utilize an LLM to estimate code correctness are shown.: quantitative and qualitative evaluation of the code generation. In the quantitative approach the LLM is prompted with the generated code and ask it to assign a numerical probability of correctness for the prompted code. The qualitative approach gives the model a scale of quality categories (see Table 3.1) and the LLM selects one of the predefined measures.

The model that was chosen as evaluator LLM is the „gpt_4o_mini“model from OpenAI. The research question for this section is:

RQ 1: How well does gpt-4o-mini estimates the correctness of the generated code?

Quantitative

First the quantitative approach is considered. Figure 4.8 shows the uncalibrated quantitative evaluation results. Uncalibrated because it is possible to calibrated the results with the investigated methods. With the blue shades the sample distribution is visualized. The most samples (500) are in the range of 85%-95%. The bin with a confidence of 95% has a correctness of only 50% and is therefore not well calibrated. Below the confidence of 55% are only a few samples and none of them are correct.

Generally the model is overconfident in its evaluation and does not achieve a well calibrated distribution. A reason for this could be that the LLM is not good in generating numerical values to express a probability.

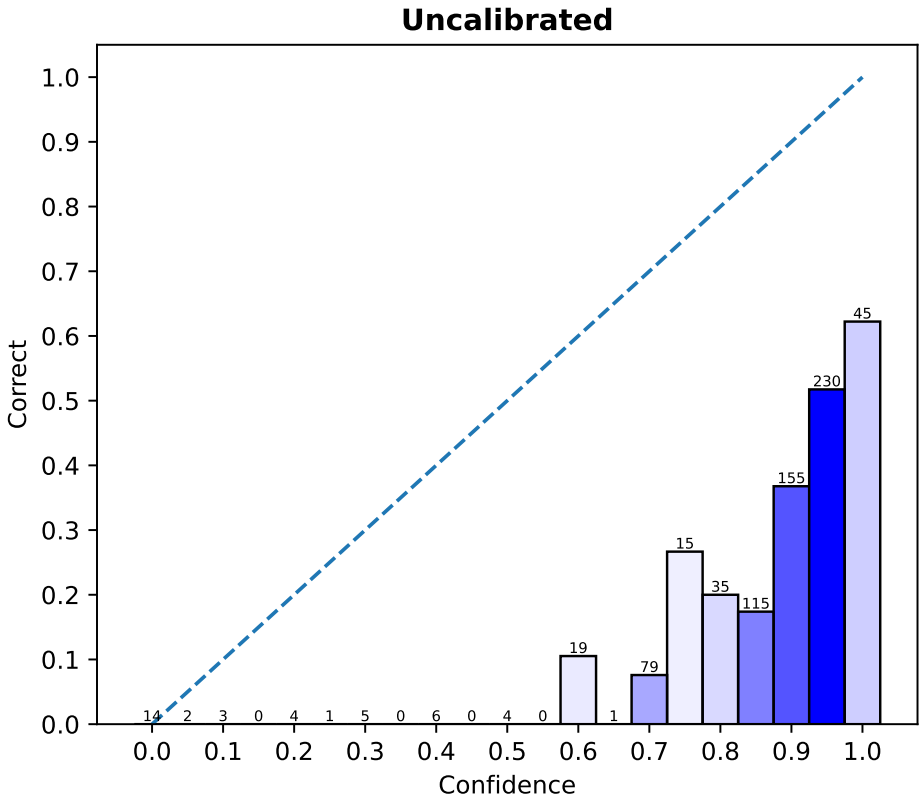


Figure 4.8: Diagram display the confidence on the x-axis and the correctness per bin on the y-axis. The dotted line represents perfect calibration. All correct samples are above the confidence of 55%. The most samples are located around the confidence of 95%. The quantitative results are not well calibrated.

Qualitative

On the other hand the qualitative approach is considered. Here use the LLM to select an option from a quality scale. Each option has a specific probability assigned. Figure 4.9 shows the reliability diagram for the qualitative approach. The samples are distributed in bins and are apart by 15%. This is the case because of the assignment with our scale values, which are shown in Table 3.1. Furthermore the higher the confidence is the higher the correctness of the bin gets. An exception is the bin 15% where no correct samples are located. The most samples are located at the confidence values of 30%/75%/90%.

Visually the calibration looks better then the calibration of the quantitative approach and the average token probability baseline. But it is still not well calibrated as the each bar is below the perfect calibration line.

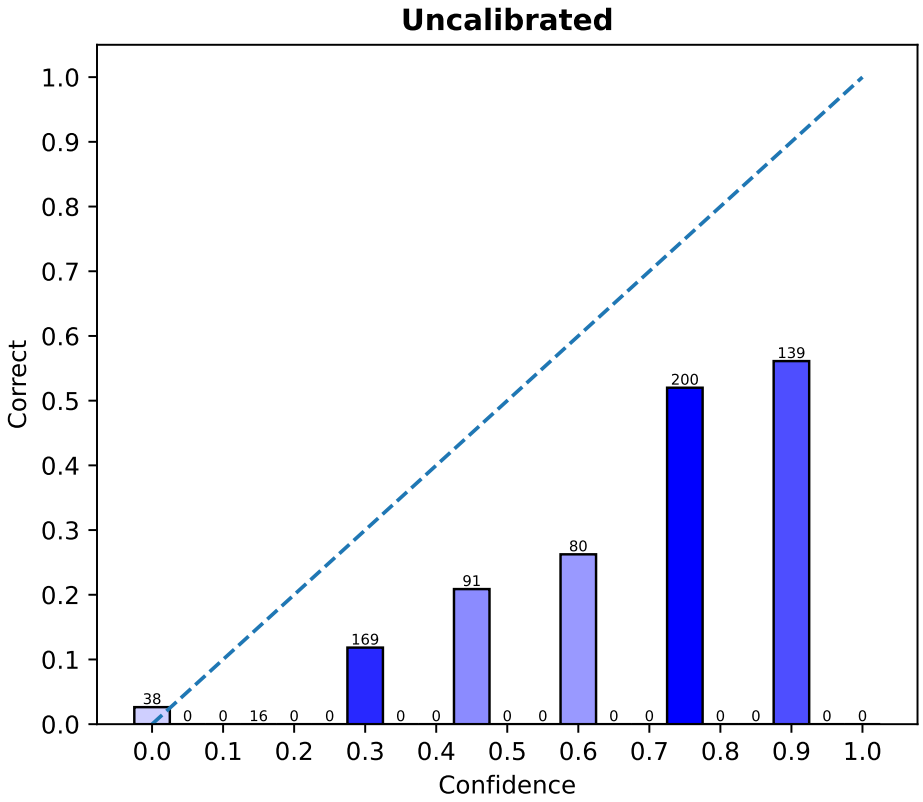


Figure 4.9: Diagram display the confidence on the x-axis and the correctness per bin on the y-axis. The dotted line represents perfect calibration. The distance between the bins is 15%. The reason for this is the qualitative scale 3.1 that was used.

Metrics

Visually the quantitative approach is better calibrated out of the box then the average token probability baseline and quantitative approach. Table 4.12 shows the metrics for the baseline and the two evaluation approaches. The quantitative approach has the highest ECE, ASCE and MSE values. Also the skill score is higher. Therefore the quantitative approach is the worst calibrated of the three. The average token probability baseline is better calibrated out of the box then the quantitative evaluation. It achieves better scores on all metrics. The best calibrated approach is the qualitative one. It achieves the best scores on all metrics. Especially the ASCE is surprisingly low.

Baseline	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Avg. Token Prob	0.376	0.151	0.355	0.222	-0.599
Quantitative	0.507	0.274	0.460	0.222	-1.072
Qualitative	**0.240**	**0.0640**	**0.246**	0.222	**-0.108**

Table 4.12: Shows the metrics for each approach and the average token probability baseline. The bold scores indicate the best value for the shown metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

In conclusion the different approaches with visualizations and metrics where shown to answer our research question RQ1. The qualitative approach is the best calibrated out of the box, But it is still worse then the MSE_{ref}. So the gpt-4o-mini estimates the correctness better then the average token probability and the best with the qualitative method.

4.5 Dataset Considerations

This experiment shows the results of our calibration approaches. All calibration approaches are tested on the original humaneval dataset (only python) and with only one generation per sample. Note that the original dataset has 164 programming tasks. Therefore 164 samples exist to work with which are splitted into train, test and validation subset. This splitting results in 33 samples for our test dataset.

> RQ 2: Is the original humaneval dataset (164 samples) enough to achieve a reasonably good calibration?

Figure 4.10 shows the uncalibrated test data in the first row. There is no clear increase in correctness from the left x-axis to the right x-axis. Most of the bins show a correctness of 50% and only have two samples. The most samples (7) are located at the confidence of 80%.

Visually none of the displayed calibration approaches achieve a good calibration. After the calibration approaches were applied the most bins have a correctness above the perfect calibration line. This means that the samples in this bin are now underestimated in their correctness. For example the bin with 30% in the histogram binning approach has a confidence of 30% but an actual correctness of around 65%.

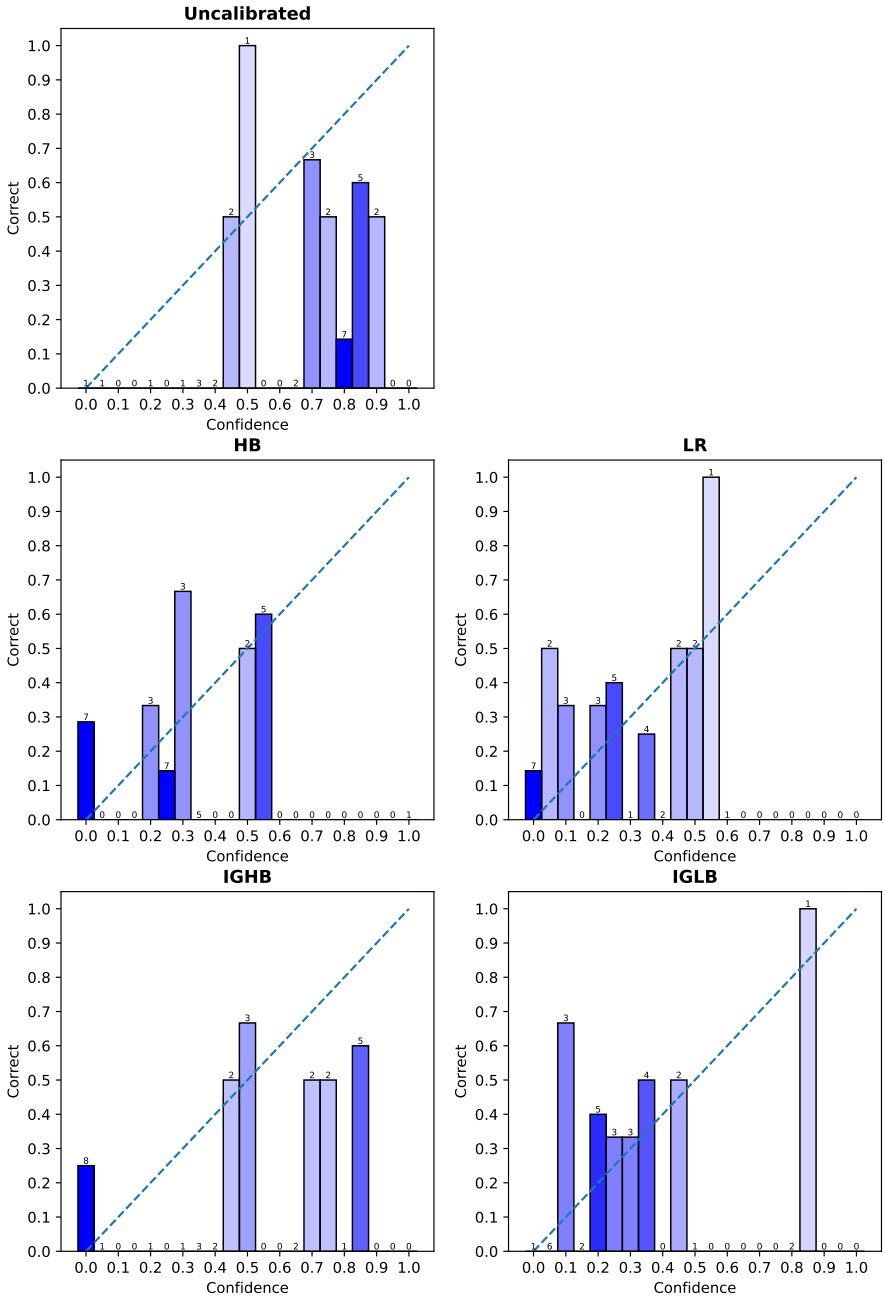


Figure 4.10: Displays the reliability charts for the avg token probability baseline and the calibration approaches. None of the approaches achieve a visually good calibration.

To be sure that the calibration did not generalize well, the calibration metrics are displayed in Table 4.13. The ECE and MSE improved slightly on all approaches. The ASCE also improved but more than the ECE. Generally the linear regression approach provides the best results for the restricted amount of data.

Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Uncalibrated	0.350	0.173	0.301	0.211	-0.427
HB	0.220	0.083	0.244	0.211	-0.156
LR	**0.194**	**0.057**	**0.228**	0.211	**-0.081**
IGHB	0.280	0.103	0.251	0.211	-0.190
IGLB	0.200	0.088	0.230	0.211	-0.090

Table 4.13: Shows the metrics for each approach that is used for the original humaneval dataset. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To summarize: visually a strong calibration improvement can not be identified. The metrics of the approaches show a gentle decrease against the baseline calibration. It is concluded that the amount of samples that are generated with the humaneval dataset is not enough to create a good calibration model. A reason for this could be the bad generalization on the small dataset, which can be seen in the reliability charts for all used calibration approaches. Only one bin per approach looks like it is well calibrated. This suggests that achieving better calibration results would require more data, even compared to our main dataset.

4.6 Grouping Criteria

In this experiment the impact of different grouping styles is evaluated. This styles were defined in Chapter 3. To recapitulate three grouping style where mentioned: the simple grouping style, complexity and categories.

- The **simple** grouping style checks if the prompt is longer than the median prompt length, if the prompt contains the word „example“and if the code is longer than the median code.
- With the **complexity** grouping style each program is classified into categories of how complex the generated program is.
- The **categories** grouping style assigns the code groups by checking for certain keywords. For example checking for the keywords like 'integer', 'float' and a combination of 'number' and 'calculate'.

The following research question arises:

RQ 3: Which grouping style provides better calibration?

Therefore the results are displayed for the complexity and categories grouping style. The results of the simple grouping style where already displayed in the previous results Section 4.3. For the sake of completeness the diagram that displays the calibration per used calibration approach for each grouping style can be found in the Appendix A.1, A.2. In the follow Section the metrics, group calibration and GASCE values for both grouping styles are displayed.

Complexity

Figure 4.7 shows the calibration for each complexity group. Each circle represents one group. The circle diameter indicates the number of samples in the group. The color assignment is as follows:

- Cyclomatic complexity: 1-10 ●
- Cyclomatic complexity: 1-20 ●
- Cyclomatic complexity: 21-50 ●
- Cyclomatic complexity: >50 ●

The groups are not well calibrated in our baseline. This is displayed in the diagram in the first row. The blue group contains the most samples, where the red group is almost invisible. All groups are located around the confidence of 70%-80%.

The histogram binning calibrated the blue and yellow group almost perfectly. The groups with a low sample count (green and orange) are not well calibrated. The linear regression calibrated the blue, yellow and green group better but also did not achieve good calibration within the red group. After both approaches the yellow and blue group have a confidence of 30%-40% and the correctness is also in that range.

Iterative group histogram binning only adjusted the values for the blue group. But the adjustment was not enough to reach the perfect calibration line. The other groups were not adjusted. The reason for this can be the low sample count and therefore low max error.

The improved iterative group linear binning did a better job calibrating the blue group. This group is well calibrated. But it also fails to correct the other groups.

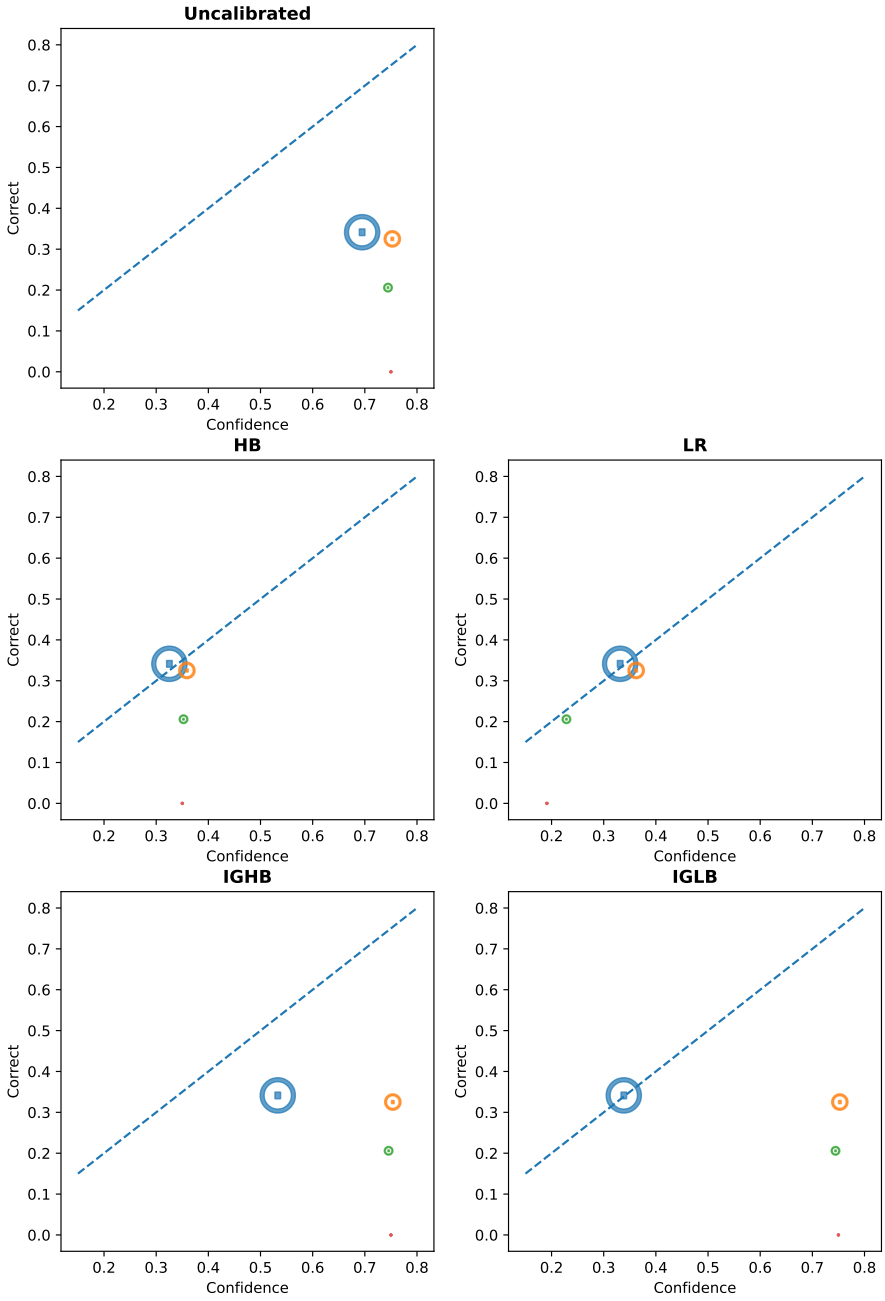


Figure 4.11: Diagram to compare the group calibration on the complexity grouping style for the baseline and each approach. Each circle represent a group. The diameter displays the number of samples in said groups. The groups are the follow: Cyclomatic complexity: 1-10 ●, Cyclomatic complexity: 1-20 ●, Cyclomatic complexity: 21-50 ●, Cyclomatic complexity: >50 ●

Categories

The group calibration results for the categories grouping style are displayed in Figure 4.12. All groups are located around the confidence of 70% in our baseline. And each group has roughly the same number of samples.

The colors of each group are assigned as follows:

- strings ●
- **not** strings ●
- mathematical ●
- **not** mathematical ●
- data objects ●
- **not** data objects ●

Figure 4.12 shows that the histogram binning only shifted the groups into a lower confidence range. That made the calibration better because the groups are now closer to the perfect calibration line. But it did not manage to adjust the red and green group to perfect calibration.

The linear regression also shifted the confidence values to the lower values, but in contrast to the histogram binning all groups have a larger distance to the perfect calibration line. This means that the groups above the line are underestimated and the groups beneath the line are overestimated in their correctness.

Iterative group histogram binning did not manage to adjust any group. The problem can be that the max error is to high with the value of 0.05.

The iterative group linear binning on the other hand did some adjustments to the groups. But every group, except the green one, is far off from being well calibrated.

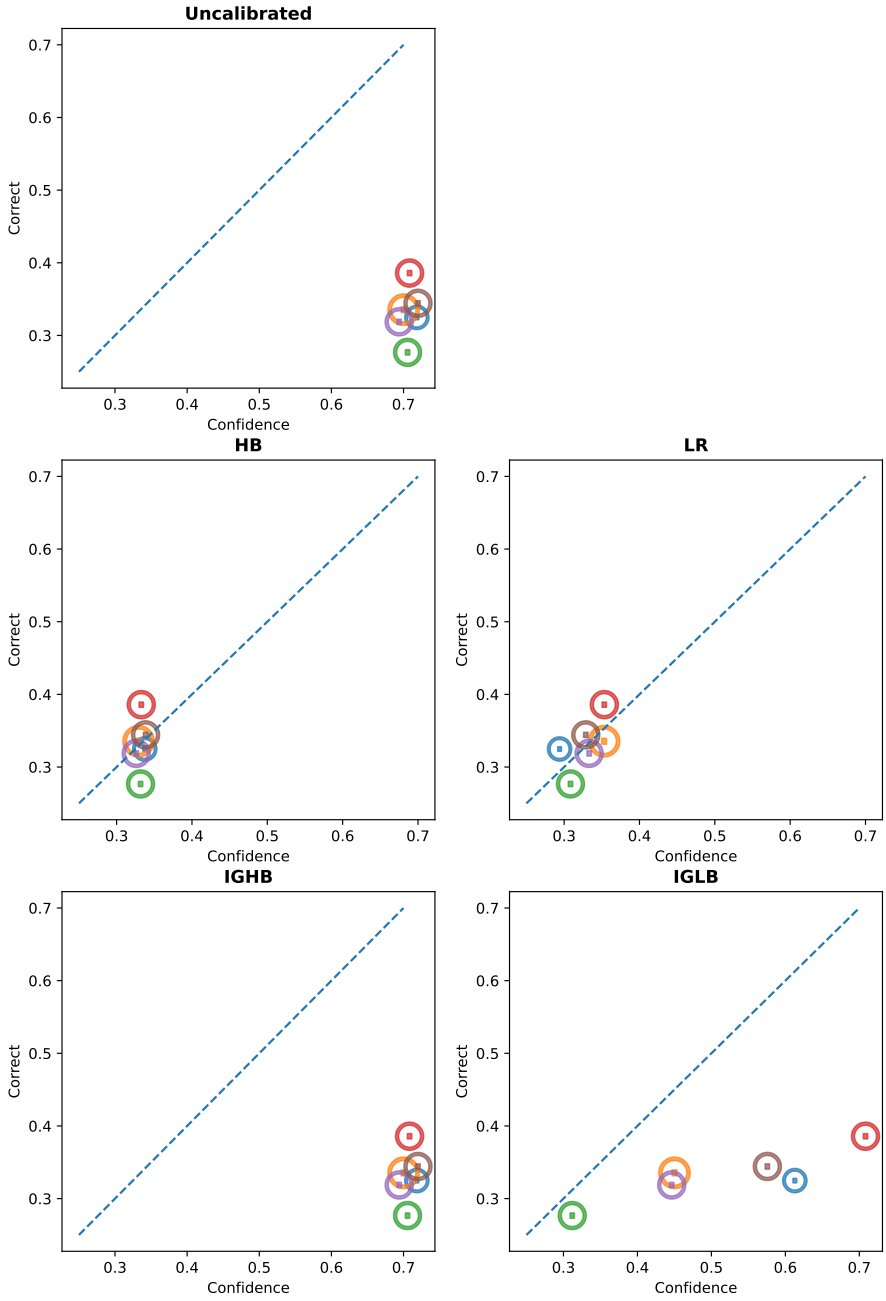


Figure 4.12: Diagram to compare the group calibration of the categories grouping style for the baseline and each approach. Each circle represent a group. The diameter displays the number of samples in said groups. The groups are the follow: strings ●, **not** strings ●, mathematical ●, **not** mathematical ●, data objects ●, **not** data objects ●

Metrics

To go into more detail the metrics for each grouping style are now shown. Table 4.14 shows these metrics. The table is ordered by the calibration approaches. The bold highlighted metrics for the grouping style show which calibration approach achieved the best score.

For the histogram binning the grouping style does not have any effect. This is not a surprise because groups were not used in this calibration approach.

The next calibration approach is the linear regression. The best ECE value was achieved with the complexity and combined grouping styles. But not far off is the simple grouping style with only a 0.04 higher ECE score. The ASCE is nearly the same on all grouping styles for the linear regression. Same goes for the MSE. Here the combined grouping style achieve the best value with 0.205 and therefore the best skill score with 0.077.

The iterative group histogram binning approach achieved best values on the simple grouping style. Nevertheless the ECE values are the same for grouping style simple and complexity.

Our last approach the iterative group linear binning achieved the best score on all metrics with the simple and combined grouping style.

This results indicate that the simple grouping style worked the best for the iterative group histogram binning and iterative group linear binning calibration approaches. The combined grouping style achieved best results with linear regression and the iterative group linear binning approach achieved on par scores with the simple grouping style. Nevertheless histogram binning achieve the best results on nearly every score, besides the MSE, where linear regression achieved a bit better results.

Grouping style	Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
	Baseline	0.376	0.151	0.355	0.222	-0.599
-	HB	**0.036**	**0.003**	0.209	0.222	**0.059**
simple	LR	0.057	**0.004**	0.207	0.222	0.068
complexity	LR	**0.053**	**0.004**	0.208	0.222	0.063
categories	LR	0.061	0.005	0.209	0.222	0.059
combined	LR	**0.053**	0.005	**0.205**	0.222	**0.077**
simple	IGHB	**0.205**	**0.068**	**0.275**	0.222	**-0.239**
complexity	IGHB	**0.205**	0.096	0.302	0.222	-0.360
categories	IGHB	0.376	0.151	0.355	0.222	-0.599
combined	IGHB	0.217	0.077	0.285	0.222	-0.284
simple	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**
complexity	IGLB	0.123	0.044	0.251	0.222	-0.131
categories	IGLB	0.186	0.061	0.260	0.222	-0.171
combined	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**

Table 4.14: Shows the metrics for each approach that is used with the different grouping styles. The bold scores indicates which grouping style provided the best results. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

The last metric to take a look at is the GASCE. This values is displayed for each group in the Table 4.15. In the Table 4.15 all groups and the association to their grouping style can be seen. The GASCE

uncalibrated columns displays the GASCE values for the average token probabilities baseline. The bold score in each row indicates that this method achieved the best GASCE for the given group.

The histogram binning approach achieved the best GASCE the most of the time. The approach is followed by the linear regression approach which achieved same results as the histogram binning or sometime better values. Iterative group histogram binning on the other hand performed worst of all four calibration approaches. The improved iterative group linear binning approach at least managed to achieve the same results as the histogram binning on three different groups.

Grouping style	**Group**	**GASCE ↓**				
		Uncalibrated	HB	LR	IGHB	IGLB
simple	> Median Prompt Len	0.09	**0.001**	**0.001**	0.042	0.022
simple	**not** > Median Prompt Len	0.065	0.005	**0.004**	0.030	0.017
simple	examples	0.038	0.003	**0.001**	0.033	0.038
simple	**not** examples	0.117	**0.003**	0.004	0.039	**0.003**
simple	> median code	0.089	**0.002**	**0.002**	0.037	0.023
simple	**not** > median code	0.065	**0.003**	0.004	0.034	0.016
complexity	1-10	0.106	**0.003**	0.005	0.052	**0.003**
complexity	11-20	0.034	**0.003**	**0.003**	0.034	0.034
complexity	21-50	0.014	**0.001**	0.002	0.014	0.014
complexity	>50	0.002	**0.**	**0.**	0.002	0.002
categories	strings	0.061	**0.002**	0.003	0.061	0.041
categories	**not** strings	0.093	**0.004**	0.006	0.093	0.027
categories	mathematical	0.098	**0.003**	0.004	0.098	**0.003**
categories	**not** mathematical	0.060	0.006	**0.005**	0.060	0.060
categories	data objects	0.078	**0.003**	0.006	0.078	0.023
categories	**not** data objects	0.077	**0.002**	0.003	0.077	0.043

Table 4.15: The table shows the GASCE for each group that is used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group. The GASCE column display the GASCE values for every calibration approach. Bold values achieved best results on the corresponding group. The first column indicates the grouping style.

With our findings from the group calibration visualization and the different evaluated metrics, the simple and combined grouping style provided the best results with our calibration approaches. Furthermore histogram binning and linear regression provided the best results for the group calibration.

4.6.1 No counter groups

The next experiment deals with the usage of counter groups. Counter groups are the exact opposite of a defined group. If a sample is not in the group „examples“it is assigned to the counter group „**not** examples“. This enables us to also target the samples which are not represented in a group and adjust their probabilities. For this experiment the **simple** grouping style is used.

The question arises:

RQ 4: Does the usage of counter groups improve the calibration results?

Table 4.16 shows the metrics with and without the usage of counter groups. The „yes “indicates that the method in the row used the counter groups. Analog the „no “means no counter groups were used. The usage of counter groups has no effect on the histogram binning and linear regression calibration approach. The scores stay the same for both variants.

The iterative group histogram binning and the iterative group linear binning on the other hand show an improvement on all scores when the counter groups are used. The ASCE has halved on both approaches.

Counter groups	Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
	Baseline	0.376	0.151	0.355	0.222	-0.599
yes/no	HB	0.036	0.003	0.209	0.222	0.059
yes	LR	0.057	0.004	0.207	0.222	0.068
no	LR	0.057	0.004	0.207	0.222	0.068
yes	IGHB	**0.205**	**0.068**	**0.275**	0.222	**-0.239**
no	IGHB	0.376	0.151	0.355	0.222	-0.599
yes	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**
no	IGLB	0.184	0.064	0.267	0.222	-0.203

Table 4.16: Shows the metrics for each approach with and without the counter groups. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

In conclusion: the counter groups had no impact on the calibration approaches histogram binning and linear regression. But the iterative group histogram binning and the iterative group linear binning achieved better results. This means that the counter groups improve the calibration results with certain calibration approaches.

4.7 Regressor

For the last experiment the input features are tweaked and it is experimented with different regressors. To recapitulate the linear regression approach 3.4.2 used the defined groups as input features and a bias b. The following used regressors will get the defined groups, the raw average token probability and a histogram, which represent the token probability distribution, as input features. The SVR was used with default settings from scikit learn and an RBF kernel. XGBoost was also used with default settings.

The question arises:

> RQ 5: Which regressor achieves the best calibration results with the extended input features?

Table 4.17 shows the metrics that each regressor achieved. The SVR performed worst on the ECE and ASCE metrics, but is better on the MSE than the XGBoost regressor. Both SVR and XGBoost have a negative skill score and therefore did not achieved better than the naive estimator MSE_{ref}.

The linear regression did perform better than the SVR and XGBoost on every metric. It even slightly outperformed the results from the histogram binning approach 4.10. Furthermore the skill score is

almost 0.1 which means the regressor achieved better calibration results than the naive estimator MSE_{ref}.

Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
LR	**0.035**	**0.002**	**0.201**	0.222	**0.095**
SVR	0.196	0.044	0.245	0.222	-0.104
XGBoost	0.180	0.043	0.251	0.222	-0.131

Table 4.17: Shows the metrics for each regressor that is used with the extended input features. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

In conclusion other regressor besides the linear regression did not perform better with regard to calibration. Furthermore the extension of the input features helped the linear regression to slightly outperform the histogram binning approach.

Chapter 5

Discussion

zu generisch (könnte so 1:1 in jeder Arbeit stehen). Konkreter?

In the last chapter the results are recapitulated. Therefore the strengths and weaknesses of the methods from the concept are discussed. After this the evaluation results and research questions are recapitulated. The last section gives also an outlook for further possible research and optimization.

5.1 Methods

The methods used in this work were: histogram binning, linear regression, iterative group histogram binning and iterative group linear binning. This section highlights the advantages and constraints of these methods.

The histogram binning is the simplest approach that adds delta values to every bin and its samples. Therefore a appropriate number of bins needs to be select. Otherwise the approach may overfits on the data and therefore does not generalize well. Furthermore the approach needs enough data to reliably improve the calibration. Since groups were not used in this approach the individual group calibration is not directly targeted.

The next approach, linear regression, is the first approach that uses the concept of groups. Therefore a viable combination of groups have to be defined to improve calibration and group calibration. Since a bias b is learned in the training phase, samples with no assigned groups can also be adjusted. This helps to further improve calibration.

for each group, linear regression...?

Iterative group histogram binning introduced adjustments on a selected bin-group combination. This approach is prone to overfitting, because it makes very specific adjustments to the dataset and therefore does not generalize well. This was countered with a 5-Fold cross validation, with the approximately same result over all folds. The approach does not generalize well.

was found to ....

The iterative group linear binning is the improved version of the iterative group histogram binning. Here multiple improvements were made to optimize calibration and prevent overfitting. Therefore a hyperparameter ϵ needs to be defined to stop adjustments on small subsets of the data and a validation subset is used to prevent overfitting. Generally speaking the improvements had a positive effect on the calibration results.

????

The created system should not be used directly. Further improvements in regards of defining better groups and testing the approaches with real world data and problems should be made.

Greift das schon den Ergebnissen vor?

5.2 Results

Discussion of Research Questions

The work found that the simpler methods histogram binning and linear regression achieved the best calibration results. This was shown on the metrics in Table 4.10. Furthermore both approaches also achieved best calibration in the defined groups (Table 4.11). This is quite surprising because the work of Detomasso et al. [6] showed that the iterative group linear binning and histogram binning achieved the best results in their scenario on the GASCE. One possible reason that the linear regression achieved better in this work could be the including of the bias. Was heißt das?

Könnte es sein, dass die Gruppen einfach keinen Mehrwert bieten, weil sie nicht informativ genug sind?

Nevertheless its surprising that the histogram binning approach achieved better calibration results on the GASCE than iterative group histogram binning and iterative group linear binning. Because histogram binning used no groups. One reasons for this could be the fact that the groups equal each other too much.

(anders ausdrücken... in terms of what?)

Furthermore different initial probabilities of correctness were displayed. This included the average token probability and an evaluator LLM that estimated the correctness of the generated code. This leads to the first research question:

> RQ 1: How well does gpt-4o-mini estimate the correctness of the generated code?

Gpt-4o-mini was used to estimate the correctness of the generated code with a qualitative and quantitative approach. The metrics from Table 4.12 showed that the qualitative approach delivered the best calibration results. Nevertheless the calibration was not better than the naive estimator. The qualitative approach could be the best, because the LLM can better express their confidence in natural language than in numbers.

The next research question dealt with the amount of data.

> RQ 2: Is the original humaneval dataset (164 samples) enough to achieve a reasonably good calibration?

Generally speaking, the amount of data that is available for calibration plays a crucial role. With only 164 samples the methods could not generalize their learned adjustments on the test dataset (33 samples). It is advised to choose a sufficient large data basis for these calibration approaches.

Furthermore different grouping criteria were explored. Therefore the following question was asked:

> RQ 3: Which grouping style provides better calibration?

To recapitulate the grouping styles were: simple, complexity, categories and all combined. Table 4.14 shows that the simple and combined grouping style achieved the best values on nearly every score. Furthermore the GASCE is the lowest with the histogram binning and linear regression approach for every chosen group.

The group subject also arise the question:

> RQ 4: Does the usage of counter groups improve the calibration results?

Therefore the counter part for each group was built and used for calibration. Table 4.16 shows that the more complex methods iterative group histogram binning and iterative group linear binning achieved better results with the usage of counter groups. Linear regression on the other hand achieved the same values with and without counter groups. That could be due to the fact that a bias was used.

The last research question deals with the usage of different regressors and the extension of the input features.

> RQ 5: Which regressor achieves the best calibration results with the extended input features?

The Table 4.17 shows that the linear regression outperformed each of the other used regressors (SVR, XGBoost). Furthermore the calibration metrics achieved slightly better score with the extended input features. ... und ist das am Ende besser als histogram binning?

5.3 Threats to Validity?

One possible threat is the selection of the number of bins. Is the number to big, the risk is high that the approaches overfit and either more data is needed or the number of bins need to be reduced. Is the number to low, adjustments to the samples are limited. Therefore a grid search was used to determine the number of bins.

Another possible threat is the creation process of groups over all possible programming languages which are available in the MultiPL-E benchmark. One reason for this is the different programming paradigms of the programming languages. Furthermore the current groups are mostly keyword matching. Which could be too superficial to get a good calibration. That is why the complexity score was used to get a grouping style that reflects the correctness more thorough.

Furthermore the amount of data is an important factor to improve the calibration. The results show that a low sample count does not allow the approaches to generalize well. Therefore a bigger dataset was used, where the calibration approaches achieved reasonably good results. Nevertheless it could be calibration improving to use even more samples.

5.4 Outlook

In this work several approaches to improve calibration were presented. However there is still room for improvement.

One improvement regards the groups. Here is further research needed on how to group code generations, so that the groups have a broader impact on the calibration. Since groups reflect the affiliation of samples a clustering approach could be used for the group generation. Therefore embeddings could be learned that can assign code generation to a certain group. Furthermore an LLM could be used to assign samples to groups. This can be realized by asking the LLM yes/no questions and assign the sample to groups with the output response. OK

Another approach could be to improve the underlay baseline. underlying This means using a more informative baseline than the average token probabilities. Maybe the token scores are not the best way to initial evaluate the calibration. An other approach involving an evaluator LLM was already shown, but this could be improved. This improvement could involve testing several other LLMs and optimize the qualitative evaluation approach. Another way of utilizing the evaluator LLM could be to ask the model if it thinks that the generated code is correct and awaiting an yes or no. Then the probability of the yes or no could be used for calibration.

Sources

Literaturverzeichnis

[1] GLENN W. BRIER. Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1):1 – 3, 1950.

[2] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. IEEE Transactions on Software Engineering, 49(7):3675–3691, 2023.

[3] Chih-Chung Chang and Chih-Jen Lin. Libsvm: A library for support vector machines. ACM Trans. Intell. Syst. Technol., 2(3), May 2011.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[5] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, page 785–794. ACM, August 2016.

[6] Gianluca Detommaso, Martin Bertran, Riccardo Fogliato, and Aaron Roth. Multicalibration for confidence scoring in llms, 2024.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, pages 611–626, 2023.

[8] Richard Oliver Lane. A comprehensive review of classifier probability calibration metrics, 2025.

[9] Fabrizio Le. How chatgpt is transforming the postdoc experience. Nature, 622:655, 2023.

[10] T.J. McCabe. A complexity measure. IEEE Transactions on Software Engineering, SE-2(4):308–320, 1976.

[11] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, Arman Cohan, and Arman Cohan. L2ceval: Evaluating language-to-code generation capabilities of large language models. Transactions of the Association for Computational Linguistics, 12:1311–1329, 10 2024.

[12] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. Proceedings of the AAAI Conference on Artificial Intelligence, 29(1), Feb. 2015.

[13] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Adv. Large Margin Classif., 10, 06 2000.

[14] Matthew Renze and Erhan Guven. The effect of sampling temperature on problem solving in large language models, 2024.

[15] Paul J. Roebber. The regime dependence of degree day forecast technique, skill, and value. Weather and Forecasting, 13(3):783 – 794, 1998.

[16] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. arXiv preprint arXiv:2402.02047, 2024.

[17] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Oh. Calibrating large language models using their generations only. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15440–15459, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[18] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs.nbsp;experience: Evaluating the usability of code generation tools powered by large language models. In Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA '22, New York, NY, USA, 2022. Association for Computing Machinery.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[20] Yuvraj Virk, Premkumar Devanbu, and Toufique Ahmed. Enhancing trust in llm-generated code summaries with calibrated confidence scores. arXiv preprint arXiv:2404.19318, 2024.

[21] Stephen J Wright. Numerical optimization, 2006.

[22] Tong Xiao and Jingbo Zhu. Foundations of large language models, 2025.

[23] Liangru Xie, Hui Liu, Jingying Zeng, Xianfeng Tang, Yan Han, Chen Luo, Jing Huang, Zhen Li, Suhang Wang, and Qi He. A survey of calibration process for black-box llms, 2024.

[24] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. High-Confidence Computing, 4(2):100211, 2024.

[25] Li Zhong and Zilong Wang. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 21841–21849, 2024.

[26] Úrsula Hébert-Johnson, Michael P. Kim, Omer Reingold, and Guy N. Rothblum. Calibration for the (computationally-identifiable) masses, 2018.

Online-Quellen

[27] BigCodeBench. Bigcodebench. [letzter Zugriff: 13. Jun. 2025].

[28] Ben Boyter. Sloc cloc and code (scc). [letzter Zugriff: 6. Jun. 2025].

[29] Thomas McCabe Jr. Software quality metrics to identify risk. [letzter Zugriff: 6. Jun. 2025].

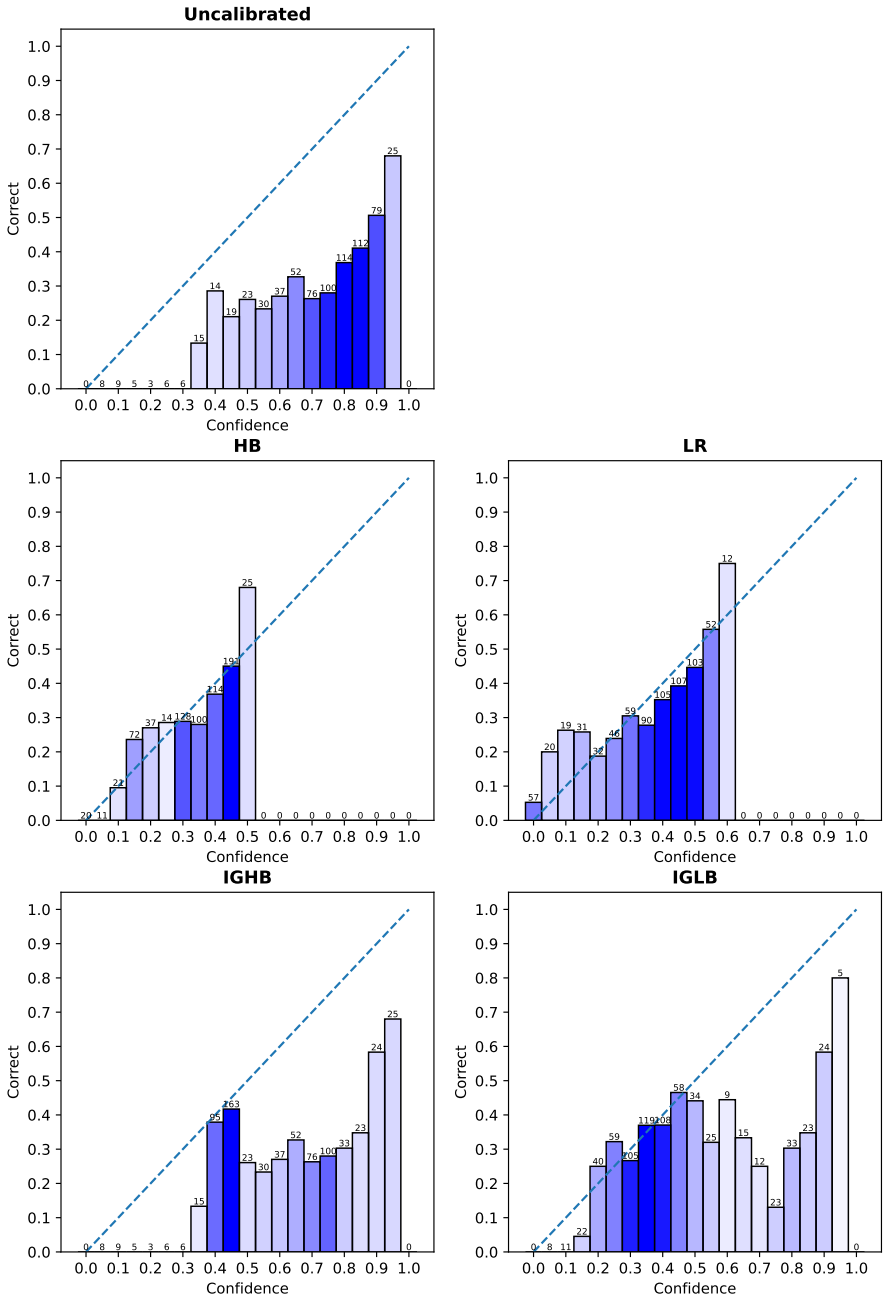
[30] Government of Canada. Probability calibration. [letzter Zugriff: 6. Jul. 2025].

[31] Amol Wagh. What's generative ai? explore underlying layers of machine learning and deep learning. [letzter Zugriff: 5. Jul. 2025].

[32] Deutscher Wetterdienst. Skill measure: Brier skill score. [letzter Zugriff: 6. Jul. 2025].

Appendix A

Charts



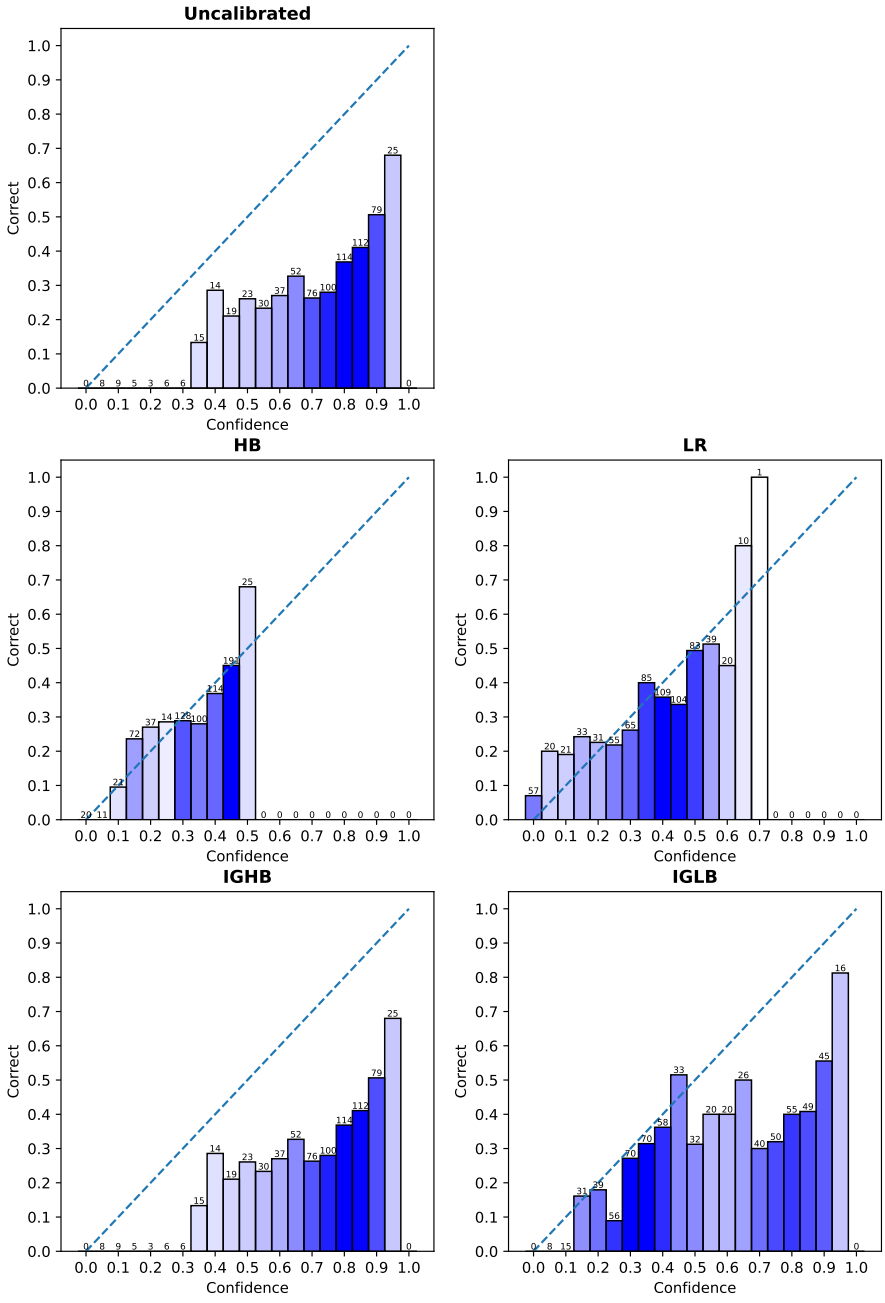

Uncalibrated
HB
LR
IGHB
IGLB
Correct
Confidence
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0

Figure A.2

Appendix B

Prompts

B.1 Evaluator LLM

"Answer as short as you can. Here is a snippet of generated code consisting of the prompt, generated code and test cases: {code}"

Verbalized quantitative prompt: "Please provide your confidence in the correctness of the provided code only in percent (0-100 %): "

Verbalized qualitative prompt: "Please provide your confidence in the correctness of the provided code only as one of {Qualitative scale:}: "

Qualitative scale: "
{
"Very low": 0,
"Low": 0.15,
"Somewhat low": 0.3,
"Medium": 0.45,
"Somewhat high": 0.60,
"High": 0.75,
"Very high": 0.9,
} "